



中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生
数学建模竞赛

学 校 _____

参赛队号 _____

队员姓名 1. _____

2. _____

3. _____

中国研究生创新实践系列大赛

“华为杯”第二十三届中国研究生数学建模竞赛

山区洪涝灾害下无人机运输与通信协同 优化

摘 要:

针对山区洪涝灾害中道路中断、地形起伏、能源受限与通信设施受损并存的应急救援场景,本文研究面向 15 个服务区、80 个不可拆货箱的无人机运输—通信协同优化问题,统一考虑地形净空、载荷相关能耗、物资时限、实体无人机与共享电池周转,以及飞行全过程连续双向通信约束,并按照“单点组批—异构运输—通信协同—任务分区”的递进思路建立模型。

针对问题一,首先依据 DEM 与机型参数计算不同服务区的最大安全载荷,并枚举单点直送条件下的可行组批模式;进一步以各类货箱剩余数量为状态,建立计数状态动态规划模型,对架次数、能耗和累计作业时间进行词典序优化。在 20% 返航安全余量下,完成全部配送至少需要 18 架次,总能耗为 59.1313 kWh;该结果在单点直送、固定航迹及本文能耗模型下为精确最优解。

针对问题二,建立考虑货箱分配、访问顺序、机型选择、实体机与共享电池调度及硬时限约束的异构多航次车辆路径模型,并采用“路线列启发式+固定路线 CP-SAT”的两层求解框架。随机化构造、局部爬山、模拟退火和禁忌搜索从匹配的独立初始方案出发,在最多 30 次有效评估及 90 s 墙钟保护下比较返航时间、能耗和架次,四法均 10/10 可行;另将历史 24 架次方案作为基线,其返航时间为 104.61 min、总能耗 71.9169 kWh,31 箱硬时限货物全部按时送达,非医疗物资加权迟到为 0。

针对问题三,为避免离散采样遗漏瞬时通信中断,利用链路距离上界与地形遮挡上界构造连续区间通信证书,并要求同一中继在整个区间内同时满足运输机接入链路和中继回程链路约束;在三类运输方案及 32 个中继窗口候选上实施有限分解协调审计。最终方案包含 26 个运输架次和 3 个中继架次,全部设备于 112.82 min 收尾,总能耗为 78.8732 kWh,共 4061 个通信区间全部通过连续可行性认证。其最小链路裕度为 0.093780 dB,而最大裕度候选仅提高至 0.094145 dB,说明有限候选内的裕度改善较小。

针对问题四,在保持问题三货箱组批、运输顺序、任务时刻及通信关系不变的条件下,将联合任务转化为不可拆任务组件,对两组和三组划分分别进行穷举,并从重复配置、库存闲置、库存缺口及工作量均衡性四方面评价独立执行代价。结果表明,两组和三组方案分别存在 3 件和 7 件资源缺口,说明任务分区虽有利于组织独立执行,却会削弱共享资源的复用效率。进一步的敏感性检验表明,最紧运输架次的水平能耗临界增幅仅为 0.5441%,现有通信证书在附加链路损耗增加约 0.1 dB 时即出现失效区间。因此,本文所得方案应视为标称参数下经过逐箱、逐航次、逐资源和逐通信区间回放验证的可执行方案,实际救援前仍需结合天气、设备状态和链路实测重新校核。

关键词: 无人机应急配送 异构车辆路径 CP-SAT 调度 连续通信证书 资源分区

目录

1. 问题重述与研究思路	4
2. 数据处理与统一物理规则	5
2.1 数据核对与空间计算	5
2.2 飞行时间和能耗	8
2.3 电池与通信状态	8
3. 模型假设及适用范围	9
4. 符号说明	10
5. 问题一：单点组批与安全载荷	10
5.1 最大安全载荷	10
5.2 不可拆货箱组批	13
6. 问题二：异构多航次运输与共享资源调度	15
6.1 时限、目标与概念槽模型	15
6.2 路线列启发式与固定路线 CP-SAT	16
6.3 固定预算下的算法对比与证据边界	16
6.4 当前可执行运输方案	17
7. 问题三：连续通信修复与有限候选分解协调	21
7.1 中继资源与覆盖约束	21
7.2 连续区间证书命题	21
7.3 有限候选分解协调审计	22
8. 问题四：固定联合任务的独立分区	28
8.1 不可拆任务组件与资源配置模型	28
8.1.1 不可拆任务组件的构造	29
8.1.2 分区空间及完全枚举	30
8.1.3 固定任务下的资源峰值计算	30
8.1.4 重复配置、库存闲置与资源缺口	31
8.1.5 词典序分区目标	32
8.2 两组与三组分区结果	32
8.2.1 两组方案的资源配置结果	34
8.2.2 三组方案的资源配置结果	35
8.2.3 重复配置、库存闲置与库存缺口比较	36
8.2.4 资源效率与工作量均衡的权衡	37
9. 敏感性与可行性检验	39
9.1 返航安全余量对单点组批的影响	39
9.2 运输能耗扰动与关键航次识别	40
9.3 通信链路附加损耗敏感性	41
9.4 主方案的全流程可行性回放	42

9.5 通信裕度候选与能源指标边界	43
10. 模型评价与结论	44
10.1 模型的主要特点	44
10.1.1 统一物理模型贯穿四个问题	44
10.1.2 根据问题结构选择不同求解层次	44
10.1.3 从目标值报告扩展到可执行性证书	45
10.2 四个问题的主要结论	45
10.2.1 问题一：单点直送与不可拆货箱组批	45
10.2.2 问题二：异构多航次运输与共享资源调度	45
10.2.3 问题三：运输-通信协同与连续链路认证	45
10.2.4 问题四：固定联合任务的独立分区	46
10.3 四问结果汇总与结论边界	46
10.4 模型的局限性	47
10.5 工程启示与最终结论	47
参考文献	49
附录 A 核心代码与复现说明	51
A.1 问题一：组批模式枚举与计数状态动态规划	51
A.2 问题二：路线搜索与固定路线排程核心代码	52
A.3 问题三：连续通信链路认证核心代码	55
A.4 问题四：不可拆组件分区与资源评价核心代码	57

1. 问题重述与研究思路

连续降雨后的山区救援同时面临道路中断、地形起伏及固定通信设施受损。新华社现场报道记录了镇龙乡道路、通信中断以及无人机投送物资的情形，新华每日电讯进一步报道了“孤岛乡”的徒步补给与救援过程 [1, 2]。题设提供镇龙乡标准场景，其中一个调度中心面向 15 个服务区配送 80 个不可拆货箱。运输机的飞行能力受载荷、装载体积、地形爬升和返航电量限制；执行任务时还须保持与固定网关或空中中继的双向通信。

问题一忽略实体机和共享电池时序，仅研究逐服务区的安全载荷和直接往返组批。问题二引入多点多架次、医疗与首批硬时限、其他物资及时性、8 架实体机及共享电池周转。问题三在此基础上要求飞行和交接全过程通信连续，并调度 2 架中继机及 6 组能源组件。问题四固定第三问任务和通信关系，将服务区划为两组、三组，核算独立执行所需资源。

四问的核心关系是逐级增加约束：第一问提供航段和组批能力基准，第二问确定物流时间表，第三问修复通信缺口并与运输调度协同，第四问评估共享资源被分割后产生的重复配置。本文始终使用相同的节点编号、坐标系、航段能耗和电池周转规则，避免跨问比较混入计算口径变化。

表 1-1 四问的核心难点、数学对象与求解方法

问题	关键数学对象	求解与验收方式
一	单点安全载荷与不可拆货箱组批	单调二分、计数状态动态规划
二	多点路线、时限与共享机体电池	四法路线搜索、CP-SAT 排程、逐箱回放
三	双向通信覆盖与运输中继时窗	有限候选协调、连续区间链路证书
四	不可拆任务组件和独立资源峰值	两组、三组穷举与库存缺口核算

图 1-1给出统一物理规则、四问求解及最终逐项核验的衔接顺序。

山区无人机运输—通信协同：递进建模与可执行性验收

四问逐层增加约束：每一问输出可被下一问继承的方案或边界



图 1-1 山区无人机运输—通信协同的递进建模与验收链

2. 数据处理与统一物理规则

人工智能辅助说明：本节及后文的附件数据分析、计算程序和初稿曾使用 OpenAI Codex 辅助（GPT-5，开发机构 OpenAI；该模型于 2025 年 8 月 7 日发布）。参赛队已对采用的建模思路、公式、代码和文字进行理解、复核与改写；下述数值均由随附程序重新计算并回代检查。

2.1 数据核对与空间计算

附件的逐箱清单共 80 行，逐箱编号均不重复，总载质量 758 kg，总体积 2.011 m³。调度中心记为 O01，服务区记为 S001 至 S015。实体运输机为 A 型 4 架、B 型 2 架、C 型 2 架；共享电池库存为 6、4、4 组。另有 R 型中继机 2 架和共享能源组件 6 组。DEM 是 EPSG:4326 坐标系的 1309×1486 个 30 米名义分辨率像元，其地理范围覆盖全部任务节点；Copernicus DEM 产品资料作为题目所列高程数据来源背景，栅格尺寸和坐标信息以实际附件为准 [3]。坐标以 WGS84 椭球计算测地距离，路线规划不使用道路数据。

DEM 地形、调度中心、服务区以及道路和水系等地理要素的相对位置见图 2-1。该图用于呈现任务空间与地形背景，实际航段能耗仍由后文给出的栅格穿越和逐段计算确定。

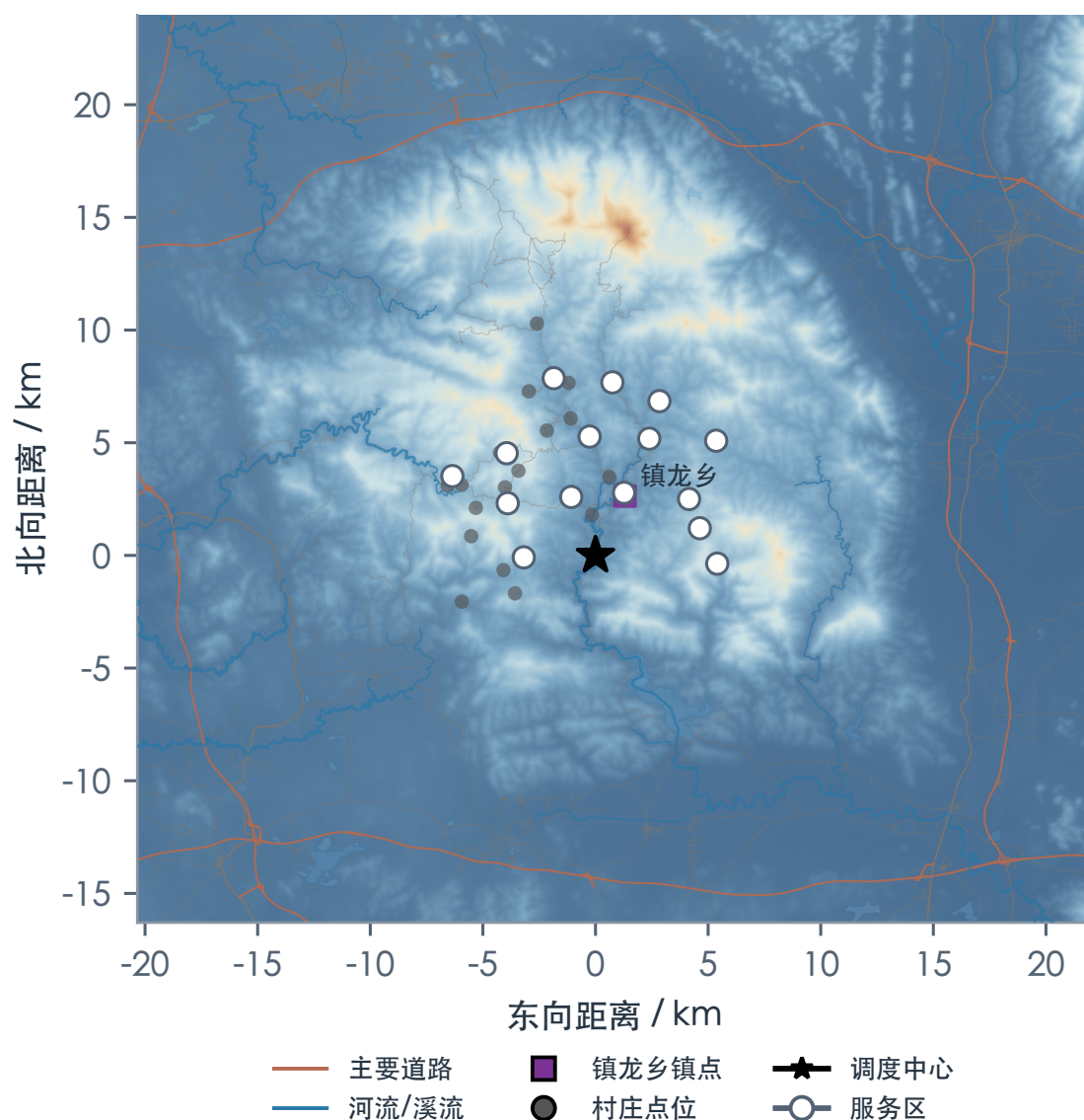


图 2-1 DEM 地形、地理要素与任务服务点分布

表 2-1 原始附件的用途及进入模型的位置

数据文件	用途
调度中心与服务区.xlsx	节点坐标、高程及需求区定位
物资需求与配送时限.xlsx	80 箱质量、体积、目的地、时限和优先系数
运输无人机数据.xlsx	A/B/C 机型能力、能量和充电参数
中继无人机数据.xlsx	R 型机、能源组件、飞行和悬停参数
通信链路参数.xlsx	双向链路预算及遮挡附加损耗
30 米 DEM 地形栅格	航段最高地形、净空与视线遮挡

80 个货箱在各服务区的类别构成不同。医疗或首批保障箱带有硬截止要求，分布见图 2-2。

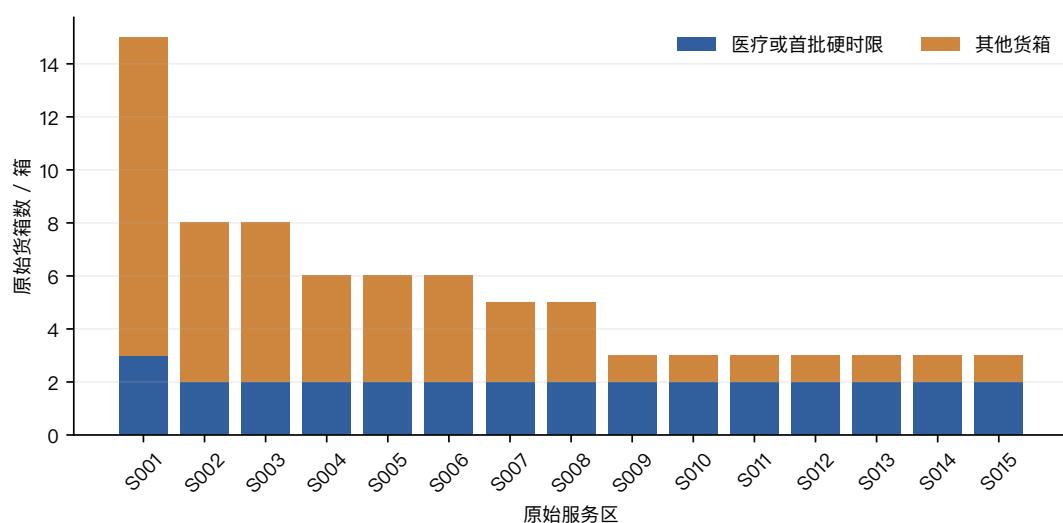


图 2-2 各服务区货箱类别与硬截止构成

表 2-2 三类运输机的核心输入参数与题给库存

机型	机体/架	电池/组	载重/kg	容积/m ³	巡航/(m/s)	单组可用电量/kWh
A	4	6	25	0.060	12	4.5
B	2	4	30	0.073	15	4.0
C	2	4	80	0.250	15	8.0

公开的专业无人机规格和无线电抗干扰产品资料可用于理解工程应用背景；例如 Matrice 350 RTK 规格页给出飞行性能指标，Doodle Labs 介绍了面向无人系统的干扰规避功能。本文的 A/B/C 机型能力、通信频率和链路门限仍完全采用题目附件，未以这些产品数据替代 [4, 5]。

不同机型与电池的题给库存是问题二实体排程和问题四独立配置的共同上限，图 2-3 将其按资源类型列出。

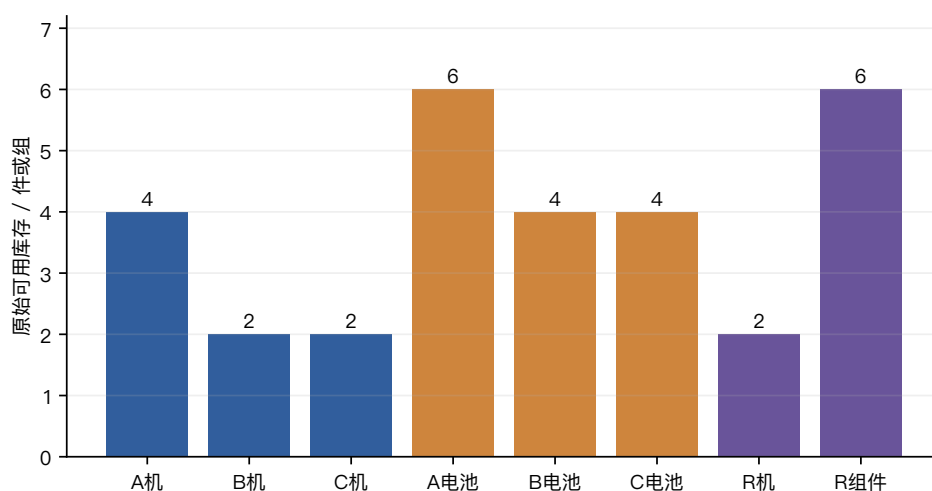


图 2-3 运输机、中继机及共享能源资源的初始库存

两节点航段水平投影采用题设直线。我们求出直线与所有经纬网格线的交点，将其分割成位于单个像元内的子段，逐段读取 DEM，令巡航海拔为沿线最高地面高程上方 50 m。这样不会因稀疏采样跳过窄山脊。调度中心的作业高度为地面高程，服务区的作业高度为地面高程上方 30 m。每一次站点交接之后重新由该站作业高度爬升。

2.2 飞行时间和能耗

对于机型 g 及起飞后当前有效载荷 q ，采用题给等效航程

$$L_g(q) = L_g^0 - (L_g^0 - L_g^F) \left(\frac{q}{Q_g} \right)^{3/2}. \quad (2.1)$$

记航段 $i \rightarrow j$ 的水平长度为 d_{ij} ，计划巡航海拔为 H_{ij} ，两端作业海拔为 z_i, z_j ，则爬升和下降高度分别为 $H_{ij} - z_i$ 与 $H_{ij} - z_j$ 。题面只写出水平与爬升能耗之和而未展开两项，本文明确采用标准航程标定的单位距离耗电和重力势能折算：

$$t_{ij}^g = \frac{H_{ij} - z_i}{v_g^\uparrow} + \frac{d_{ij}}{v_g^c} + \frac{H_{ij} - z_j}{v_g^\downarrow}, \quad (2.2)$$

$$E_{ij}^g(q) = E_g^{\text{use}} \frac{d_{ij}}{L_g(q)} + \frac{(m_g + q)9.81(H_{ij} - z_i)}{3.6 \times 10^6 \eta_g}. \quad (2.3)$$

其中 E_g^{use} 是单组电池可用能量， m_g 为含电池空载质量， η_g 为爬升能耗效率。下降不另计附加能耗。每个架次的航段能量之和须不超过 $(1 - \rho_g)E_g^{\text{use}}$ 。多点架次在某站交接后立即扣去该站箱重，再计算下一航段。

两篇参考文献为载荷相关能耗提供了物理解释和适用边界。Dorling 等由多旋翼悬停诱导功率推得

$$P_{\text{hov}}(m) = (W + m)^{3/2} \sqrt{\frac{g^3}{2\rho_a \varsigma n}}, \quad P_{\text{hov}}(m) \approx \alpha m + \beta, \quad (2.4)$$

其中 W 为机架质量， m 为电池与载荷质量， n 为旋翼数， ς 为单个旋翼盘面积。对其 Hexa-B 样机，在 $m = 0\text{--}3 \text{ kg}$ 范围内拟合得 $\alpha = 46.7 \text{ W/kg}$ 、 $\beta = 26.9 \text{ W}$ ；该文还用飞行测试检验了悬停功率对飞行功率的近似。Zhang 等综述的无风稳态平飞升阻比模型则给出单位距离能耗

$$E_{\text{pm}} = \frac{(m_{\text{body}} + m_{\text{battery}} + m_{\text{payload}})g}{r\eta} + \frac{P_{\text{avio}}}{v_a}, \quad (2.5)$$

其中 r 为升阻比， η 为电池至推进系统的能量传递效率， P_{avio} 为航电功率， v_a 为空速。该综述比较的不同模型即使在相近条件下也会产生明显不同的能耗估计，并指出爬升、下降、悬停、风和航电会改变整次任务的能量需求 [6, 7]。因此，这些公式支持“载荷影响能耗”及按飞行阶段核算的建模方向，但其样机系数、旋翼几何和巡航参数无法直接移植到题目给出的三种机型。本文水平能耗仍按题给各机型的载荷—等效航程曲线计算，爬升另按重力势能折算；参考公式不作为未标定参数的替代来源。

准备与逐箱装载时间计入起飞前占用；站点交接时间是“基础时间 + 逐箱时间”，本站所有箱的交付时刻统一记为交接结束。这一做法对时间窗保守。题设没有运输机交接悬停功率和准备工位数量，因此没有增设未经给定的能耗或工位容量参数。

2.3 电池与通信状态

任何电池或中继能源组件均初始满电。一次任务结束后 SOC 为 $1 - E/E^{\text{use}}$ ；再次装机前须按题给两阶段线性充电模型恢复 100%。锂离子电池控制资料展示了恒流、恒压分阶段充电的一种工程实现，可作一般技术背景；本文充电阶段、比例和时长仍以题目参数为准

[8]。使用、充电两个区间均占用该组件，且同类电池不可跨机型调配。运输机和中继机的实际架次时间区间亦不得重叠。

固定网关与运输机、中继机之间的双向链路门限分别为 122、126 dB，运输机与中继机之间为 116 dB。视线受 DEM 遮挡时在自由空间损耗上增加 10 dB，而非直接判为中断。每一时刻运输机必须满足网关直连，或同时满足接入中继与中继回传两条链路。自由空间损耗使用题给的 2400 MHz 公式，并以实际三维距离计算 [9]。无人机辅助无线通信也被研究为基础设施覆盖不足时的通信方式；本文则通过题给链路预算和直连/中继约束进行具体判定 [10]。

三类双向链路的允许损耗门限不同，运输机到中继链路的门限最低。图 2-4 汇总题给链路预算，后续通信判定均按对应链路类型分别核验。

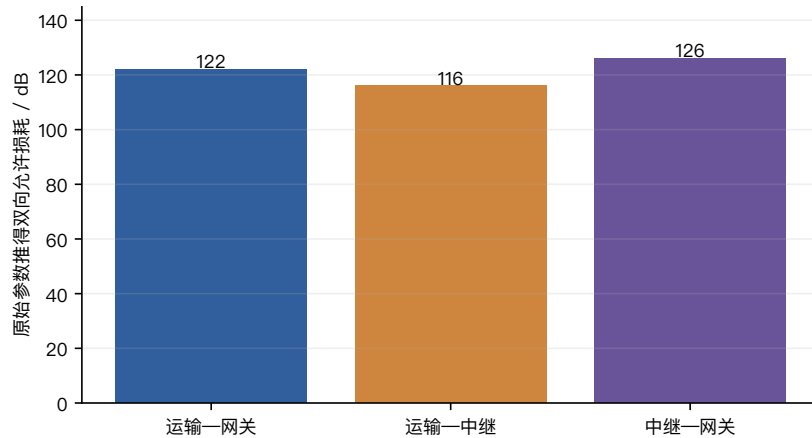


图 2-4 运输机与中继通信链路的允许损耗门限

后续求解严格区分概念模型和实现证据：问题二由路线列启发式生成完整箱组与站序，再由固定路线 CP-SAT 安排实体资源；问题三以连续区间证书修复通信可行性，并只在归档的有限运输族和中继窗口上协调。因而“可回放可行”与“全空间最优”在表述中始终分开。

3. 模型假设及适用范围

题设参数按确定值处理。无人机沿两节点水平直线飞行，巡航高度由 DEM 净空决定；不考虑风场、临时禁飞区与地面道路通行。不可拆货箱是运输的最小单位，一箱只能在原始需求表指定的服务区交付一次。第一问每架次仅服务一个服务区；后两问允许同架次多点交付，并按交付后的剩余箱重更新能耗。

准备和装载从架次开始时占用实体机及电池；返航结束后实体机可开始下一次准备，电池须先充满。附件未规定共享工位数量，因此不额外施加工位互斥。中继在给定悬停点保持不动。每次任务由 O01 出发并返回，服务前完成准备和建链；悬停和通信附加功率依附件计入能源消耗。中继任务结束与下一次准备之间保留给定的 300 s 周转时间。

全部 80 箱以完整交接结束时刻计实际送达。医疗物资须满足附件期望送达时刻，首批保障货箱须满足首批截止；兼属两类时同时满足。64 箱非医疗物资的期望时刻用于统计迟到和配送及时性，不作为不可行判据，其中 15 箱首批饮用水还受首批截止约束。题设 DEM 按地形表面使用；其标称高程、节点高程与通信设备高度视为同一垂直基准。若存在定位误差或未表示的新障碍，需重新核算航线和通信裕量。

关于目标权衡，本文先检验全部目的地、医疗及首批时限、安全、资源及通信硬约束，再按优先加权迟到、全部设备最后返航时间和优先加权交付时刻逐级比较，同时报告任务

耗电和架次数。有限候选搜索所得“相对最优”是经回代验证的可行改进，不代表全空间全局最优；标称可行也不等同现场天气下的稳健可行。

4. 符号说明

表 4-1 集合、参数与状态量

符号	含义	单位
$\mathcal{B}, \mathcal{N}, \mathcal{G}$	货箱、服务区与运输机型集合	—
$\mathcal{M}, \mathcal{H}$	医疗箱与适用硬时限货箱集合	—
$n(b), m_b, V_b$	货箱目的地、质量与体积	—、kg、m ³
d_b, f_b, h_b, D_b	期望、首批、适用硬截止与交付完成时刻	s
$\pi_b, T_b, T_\pi, \bar{D}_\pi$	优先系数、迟到量、加权迟到与加权平均交付	—、s、系数·s、s
$Q_g, \bar{V}_g, E_g^{\text{use}}, \rho_g$	载重、容积、可用电量与返航储备率	kg、m ³ 、kWh、—
$\mathbf{n}_i, \mathbf{a}, \mathcal{A}_i$	箱型计数、批次模式与可行模式集合	箱、箱、—
$F_i(\mathbf{r})$	完成剩余计数的最优（架次，能耗）	架次、kWh
$D_{ab}(t), L_{ab}(t), L_{a \leftrightarrow b}^{\max}$	三维距离、路径损耗与双向允许损耗	km、dB、dB
$\bar{L}_I, \mu_I, \gamma_L$	区间损耗上界、证书裕度与最小链路裕度	dB

表 4-2 决策变量与评价指标

符号	含义	单位
$a_k, x_{bk}, h_{ki}, v_{kij}$	概念航次启用、箱分配、站点访问与路径弧	0–1
y_{kg}, z_{ku}, w_{kr}	航次的机型、实体机与电池分配	0–1
s_k, C_k, E_k	起飞、返航时刻与航次能耗	s、s、kWh
$r \in \mathcal{R}, \lambda_r$	实际路线列及未来列选择变量	—、0–1
C_T, E_T, N_T, γ_E	运输收尾、能耗、架次与最小能源裕度	s、kWh、架次、—
$\varepsilon_T, \mathcal{P}^*, S^{\text{arc}}$	容忍量、非支配集与共享历史初值	系数·s、—、—
$e_\ell, q_{\ell h}, z_{\ell u}^R, w_{\ell r}^R$	中继启用、站点、实体机与组件分配	0–1
$[a_\ell, b_\ell], d_{km}, c_{kml}$	中继服务窗、直连和中继覆盖分配	s、0–1、0–1
$\kappa^{\text{dir}}, \kappa^{\text{acc}}, \kappa^{\text{back}}$	连续直连、接入与回程证书	0–1
C_J, E_R, N_R	联合收尾、中继能耗与中继架次	s、kWh、架次
$a_{ch}, r_{hk}, r_k^{\text{pool}}, A_k$	组件分组、组需求、统一池需求与库存	0–1、件
$R_k^{\text{dup}}, I_k^{\text{idle}}, G_k^{\text{short}}$	重复配置、库存闲置与库存缺口	件
W_h, CV_W	组工作量及其变异系数	s、—

5. 问题一：单点组批与安全载荷

5.1 最大安全载荷

单点往返架次在去程携带 q ，返程空载。固定服务区 i 与机型 g 后，安全载荷是以下一维单调约束的最大可行 q ：

$$q_{ig}^{\max} = \max \{q \in [0, Q_g] : E_{0i}^g(q) + E_{i0}^g(0) \leq (1 - \rho_g)E_g^{\text{use}}\}. \quad (5.1)$$

左端随 q 单调不减，故用二分法求解，并再受额定质量与体积约束。若空载往返已经超出能量上限，定义安全载荷为 0 且该机型对该区不可用。图 5-1 展示 20% 安全余量下的 45 个机型—服务区载荷上限，差异主要来自水平距离、沿线最高地形和各机型满载航程。

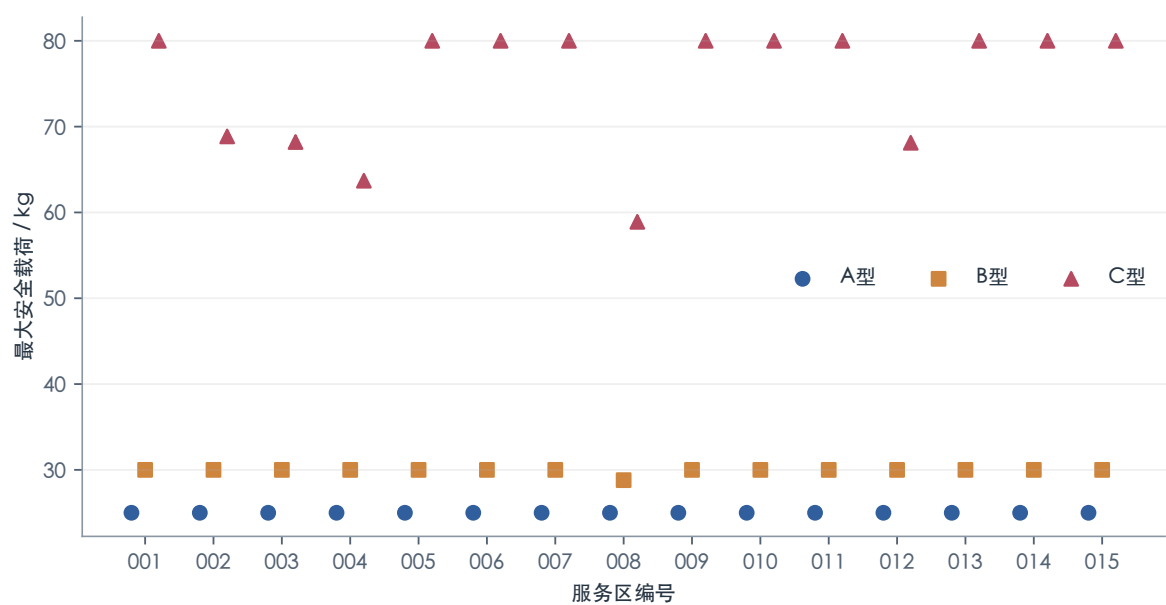


图 5-1 20% 返航安全余量下的逐区最大安全载荷

将机型和服务区同时展开后，安全载荷差异更易比较；图 5-2 以矩阵色阶呈现同一组 45 个机型—服务区组合结果。

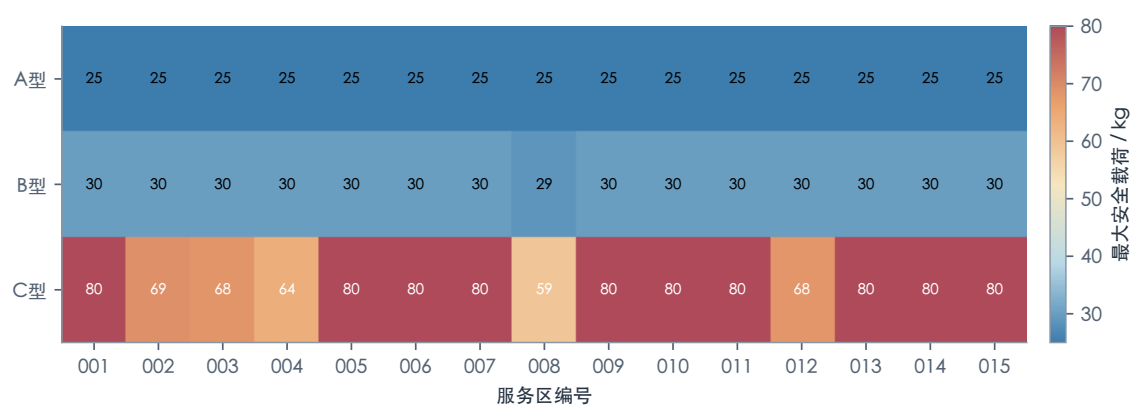


图 5-2 不同机型与服务区组合的最大安全载荷热力图

表 5-1 列出全部 45 个安全载荷数值，并汇总 20% 余量下各区实际组批数与耗电；额定载荷截断的数值仍须满足体积上限。

表 5-1 20% 返航储备下各服务区三机型最大安全载荷与单点组批

服务区	A/kg	B/kg	C/kg	箱数	架次	能耗/kWh
S001	25.00	30.00	80.00	15	2	5.913
S002	25.00	30.00	68.86	8	2	8.435
S003	25.00	30.00	68.21	8	2	8.545
S004	25.00	30.00	63.70	6	1	6.153
S005	25.00	30.00	80.00	6	1	4.222
S006	25.00	30.00	80.00	6	1	2.598
S007	25.00	30.00	80.00	5	1	3.411
S008	25.00	28.80	58.90	5	1	5.836
S009	25.00	30.00	80.00	3	1	2.096
S010	25.00	30.00	80.00	3	1	1.823
S011	25.00	30.00	80.00	3	1	1.074
S012	25.00	30.00	68.12	3	1	2.752
S013	25.00	30.00	80.00	3	1	1.825
S014	25.00	30.00	80.00	3	1	2.140
S015	25.00	30.00	80.00	3	1	2.307

安全载荷上限同时受地形爬升和载荷相关等效航程影响。图 5-3展示代表性航段沿线地形与巡航高度，图 5-4展示载荷增加时航程和能耗的变化。

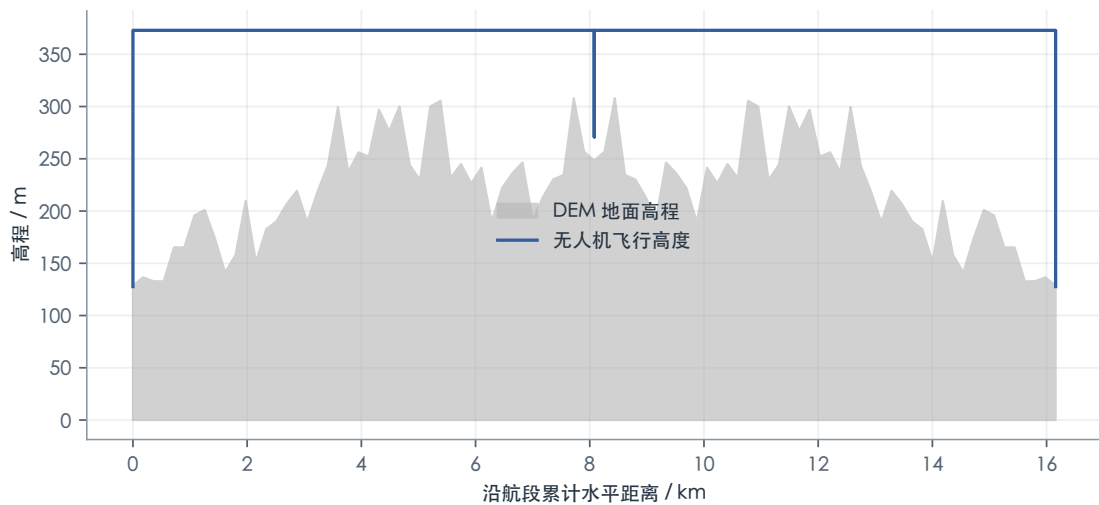


图 5-3 代表性往返航段的 DEM 高程剖面与巡航高度

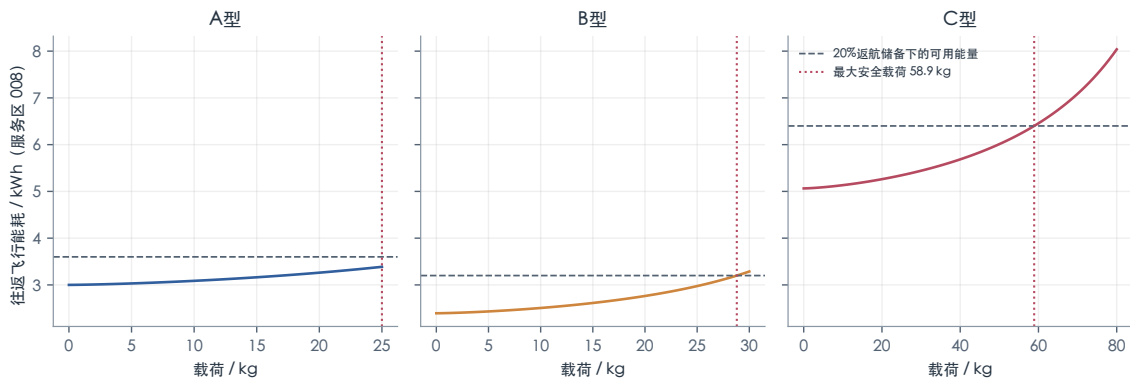


图 5-4 典型服务区不同载荷下的等效航程与飞行能耗

5.2 不可拆货箱组批

令 B_i 为服务区 i 的货箱集合, $a \subseteq B_i$ 为单个候选组批。候选组批的箱重、体积和往返能耗必须分别满足 $Q_g, V_g, (1 - \rho_g)E_g^{\text{use}}$ 上限。通过 $\sum_{a \ni b} x_a = 1$ 保证每箱且仅一架次, 并限制 $a \subseteq B_i$ 防止跨区组批。目标按总架次数、总能耗、累计架次作业时间字典序最小化。

由于同服务区同种物资的货箱具有相同质量、体积和飞行代价, 用各物资类型的剩余箱数构成动态规划状态。转移枚举所有可行装载计数组合及机型; 终止状态为零箱。恢复最优转移后按原始货箱编号分配, 因此没有把整箱货物拆开。这个有限状态递推覆盖了全部可行分组, 给出第一问所述字典序下的全局最优值。对应核心代码见附录 A.1。

具体地, 令 n_i 为服务区 i 的箱型计数, $\mathcal{A}_i$ 为满足载重、容积和能源约束的全部非零机型-批次模式。定义 $F_i(\mathbf{r})$ 为完成剩余计数 $\mathbf{r}$ 的最优 (架次, 能耗), 则

$$F_i(\mathbf{0}) = (0, 0), \quad F_i(\mathbf{r}) = \min_{\text{lex}} \{F_i(\mathbf{r} - \mathbf{a}) + (1, E_{gi}(\mathbf{a})) : (g, \mathbf{a}) \in \mathcal{A}_i, \mathbf{a} \leq \mathbf{r}\}. \quad (5.2)$$

无可行转移时取 $(+\infty, +\infty)$ 。对 $\sum_k r_k$ 作归纳即可得到 Bellman 最优性; 因此结论只在单点直送、固定轨迹和本文能耗模型内成立, 不外推到多站实体排程。

表 5-2 返航安全余量改变时的组批结果

安全余量	最少架次	能耗/kWh	累计作业时间/s	状态
10%	18	59.1313	32776.02	可行
20%	18	59.1313	32776.02	可行
30%	20	67.2276	36586.18	可行
40%	—	—	—	S004 的 14 kg 水箱不可直送

20% 余量下, 主方案为 18 架次、59.1313 kWh。若放弃最少架次而只优化能耗, 则为 19 架次、59.0339 kWh, 仅节省 0.0974 kWh, 却增加 1 架次及 1663.96 s。故在应急任务中先压缩架次更合理。安全余量升至 30% 时最少架次增为 20; 40% 时 S004 的 B 型最大安全载荷仅 8.6665 kg, 其余机型无法完成安全空载往返, 14 kg 整箱饮用水无可行机型。图 9-1 给出可行区间内的变化。

最少架次方案与最少能耗方案的时间—能耗取舍如图 5-5 所示; 主方案逐架次的返航电量及安全储备线见图 5-6。

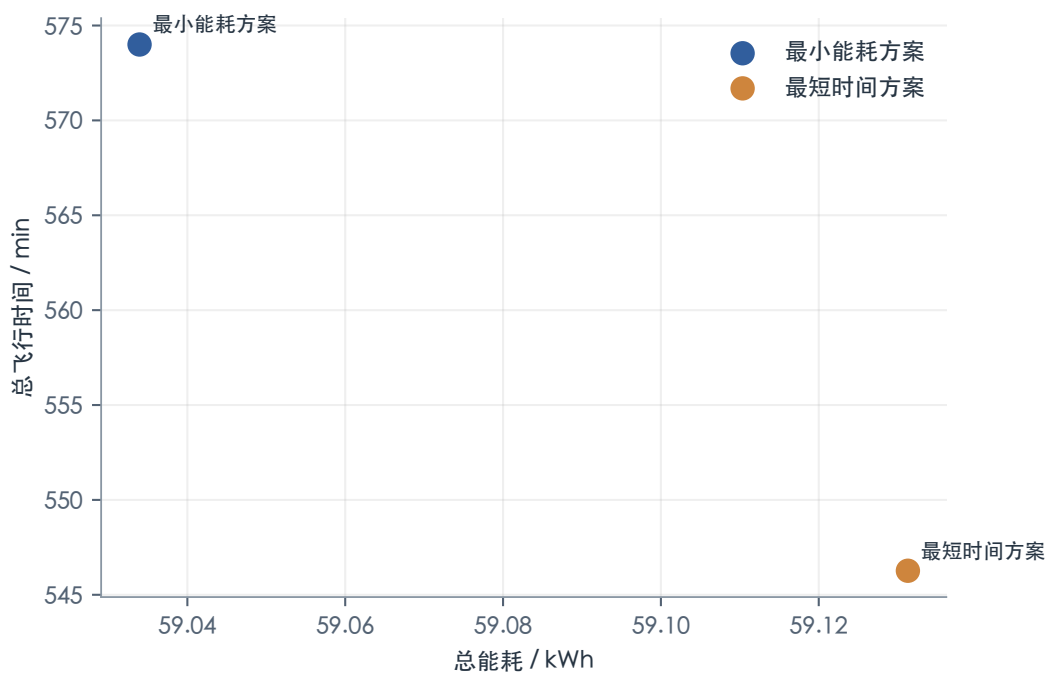


图 5-5 问题一时间与能耗单目标方案比较

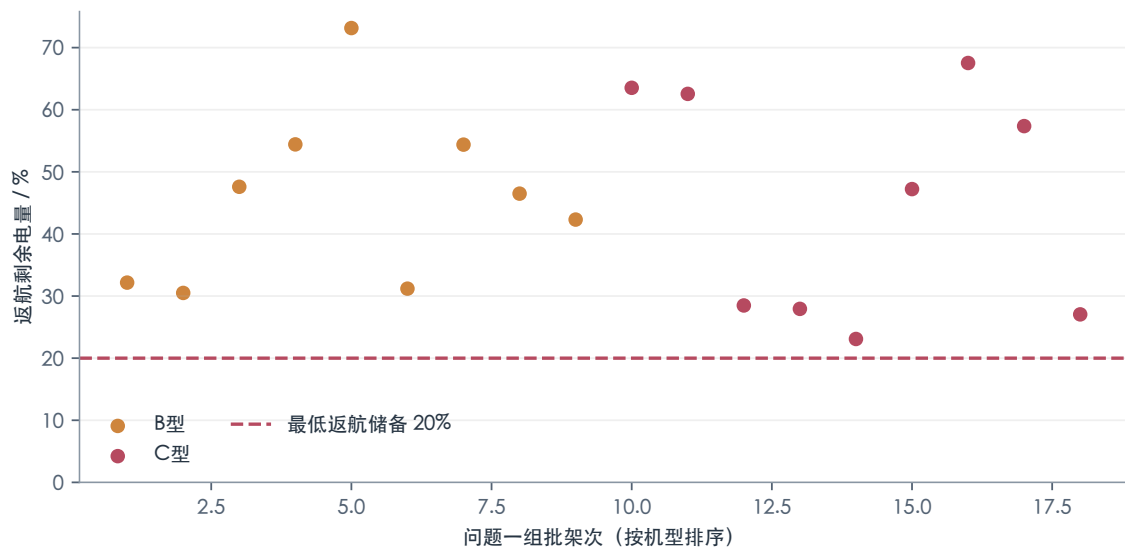


图 5-6 问题一主方案逐架次返航剩余电量

6. 问题二：异构多航次运输与共享资源调度

6.1 时限、目标与概念槽模型

航次 k 从 O01 出发并返回，货箱 b 只能在原始目的地 $n(b)$ 交接一次。其适用硬截止为

$$h_b = \begin{cases} \min(d_b, f_b), & b \text{ 兼属医疗与首批}, \\ d_b, & b \text{ 仅属医疗}, \\ f_b, & b \text{ 仅属首批}, \\ +\infty, & \text{其他}. \end{cases} \quad D_b \leq h_b \quad (b \in \mathcal{H}). \quad (6.1)$$

共 31 箱受硬截止约束；非医疗箱的期望时刻只进入 $T_b = \max(0, D_b - d_b)$ 。概念完整模型用 $a_k, x_{bk}, h_{ki}, v_{kij}$ 分别表示航次启用、箱分配、站点访问和路径弧，并用 y_{kg}, z_{ku}, w_{kr} 分配机型、实体机和电池：

$$\sum_k x_{bk} = 1, \quad x_{bk} \leq a_k, \quad x_{bk} \leq h_{k,n(b)}, \quad h_{ki} \leq \sum_{b:n(b)=i} x_{bk}, \quad (6.2)$$

$$\sum_{j \in \mathcal{N}} v_{k0j} = a_k = \sum_{i \in \mathcal{N}} v_{kij}, \quad \sum_j v_{kij} = h_{ki} = \sum_j v_{kji}, \quad (6.3)$$

$$o_{ki} - o_{kj} + |\mathcal{N}|v_{kij} \leq |\mathcal{N}| - 1 + |\mathcal{N}|(2 - h_{ki} - h_{kj}), \quad (6.4)$$

$$\sum_g y_{kg} = a_k, \quad \sum_{u \in \mathcal{U}_g} z_{ku} = y_{kg}, \quad \sum_{r \in \mathcal{R}_g} w_{kr} = y_{kg}. \quad (6.5)$$

载重、容积与逐段载荷能耗满足

$$\sum_b m_b x_{bk} \leq \sum_g Q_g y_{kg}, \quad \sum_b V_b x_{bk} \leq \sum_g \bar{V}_g y_{kg}, \quad E_k \leq \sum_g (1 - \rho_g) E_g^{\text{use}} y_{kg}. \quad (6.6)$$

令 $C_k = s_k + \Delta_k(x, v, y)$ ，当 $x_{bk} = 1$ 时强制 $D_b = s_k + \tau_{kb}(x, v, y)$ 。对每架实体机设置存在变量为 z_{ku} 的可选飞行区间 $[s_k, C_k)$ ；对每组电池设置存在变量为 w_{kr} 、延长至充满的可选区间。分别施加 **NoOverlap**，从而闭合机体、电池和充电周转约束。该结构属于带时间窗的异构多航次车辆路径问题，其经典路由基础、时间窗算法和 VRP 方法可分别参照 Dantzig 与 Ramser、Solomon 及 Toth 与 Vigo[11, 12, 13]。无人机配送研究还包括卡车—无人机协同路线及相关综述；这些工作与本文多架无人机从固定中心独立出发的设置不同，仅作为问题背景[14, 15]。模拟退火和禁忌搜索的算法思想及其在车辆路径问题中的应用可参照 Kirkpatrick、Glover 和 Osman[16, 17, 18]。

评价量为

$$T_\pi = \sum_{b \notin \mathcal{M}} \pi_b T_b, \quad C_T = \max_k C_k, \quad \bar{D}_\pi = \frac{\sum_b \pi_b D_b}{\sum_b \pi_b}, \quad E_T = \sum_k E_k. \quad (6.7)$$

先最小化 T_π ，再加 $T_\pi \leq T_\pi^* + \varepsilon_T$ ，依次考察 $C_T, E_T, N_T, -\gamma_E$ ；其中

$$\gamma_E = \min_{k:a_k=1} \frac{(1 - \rho_{g_k}) E_{g_k}^{\text{use}} - E_k}{E_{g_k}^{\text{use}}}. \quad (6.8)$$

不同容忍量对应非支配集 $\mathcal{P}^*$ ，而非把指标任意加权成一个分数。

6.2 路线列启发式与固定路线 CP-SAT

完整槽模型用于说明变量与约束，实际求解采用两层结构。上层搜索完整路线列 $r = (B_r, \sigma_r, g_r, E_r, \Delta_r, \{\tau_{rb}\})$ 的覆盖；下层在箱集、站序和机型固定后，只由 CP-SAT 决定起飞时刻以及机体、电池分配 [19]：

$$C_r = s_r + \Delta_r, \quad D_b = s_r + \tau_{rb}, \quad \sum_{u \in \mathcal{U}_{g_r}} z_{ru} = 1, \quad \sum_{q \in \mathcal{R}_{g_r}} w_{rq} = 1. \quad (6.9)$$

本文没有实现列生成主问题，故不把实际路线列 r 混称为概念槽 k ，也不宣称求得完整 VRPTW 的全局最优。

局部爬山、模拟退火和禁忌搜索共享五类真实邻域：调整航次次序、整箱跨航次搬移、单箱拆成直送、更换机型或重排站序、合并两个航次；每次最多尝试 35 次生成物理可行变换。随机化构造则独立重建方案，不称作共享五邻域。爬山每轮考察 10 个邻居、连续 8 轮无改进回到当前最好解；禁忌每轮考察 10 个邻居、禁忌期限 12 轮并设优于全局最好解的破禁规则；退火最低温度 0.001，温度随已评估比例线性下降，词典序相对损失按 0.02 尺度进入接受概率。所有数值均来自实际程序配置，而非事后调参。该随机化构造会扰动优先级和评分参数，再以贪心准则逐步扩展路线；GRASP 文献可作为随机贪心构造的一般方法背景，但本文不将该实现称为标准 GRASP[20]。

6.3 固定预算下的算法对比与证据边界

正式协议对每个种子先由同一随机化构造生成一个独立的硬可行初始解，四种方法共享该初始解；随后在相同的有效候选数上限（30 次）和 90 s 墙钟保护下独立搜索。每个候选均由同一资源解码器按实体机与电池的可用时刻排程并计算指标；各方法得到的最佳路线计划再交给同一固定路线 CP-SAT，依次压低加权迟到、收尾时刻和加权交付时刻。这样可以在相同数据、约束和初始条件下比较返航时间、能耗与架次。每种方法运行 10 个种子，四法均为 10/10 可行，且 40 次运行均完成 30 次评估、未触发墙钟保护；表 6-1 报告均值和总体标准差。随机化构造、爬山、退火和禁忌分别在 9、10、9、10 次运行中找到严格改善。核心代码见附录 A.2。

表 6-1 同初始解下四种路线搜索的性能比较（均值 $\pm$ 标准差，每法 10 个种子）

方法	可行运行	返航/min	能耗/kWh	架次
随机化构造	10/10	159.19 $\pm$ 17.12	76.7629 $\pm$ 2.1321	23.70 $\pm$ 0.90
局部爬山	10/10	142.13 $\pm$ 13.11	80.7906 $\pm$ 3.9916	26.70 $\pm$ 1.19
模拟退火	10/10	148.74 $\pm$ 13.57	82.2837 $\pm$ 5.2026	27.40 $\pm$ 2.06
禁忌搜索	10/10	141.51 $\pm$ 13.53	81.3108 $\pm$ 3.6084	27.00 $\pm$ 1.00

独立起点协议下，随机化构造、爬山、退火和禁忌的返航时间均值分别为 159.19、142.13、148.74 和 141.51 min，能耗均值分别为 76.7629、80.7906、82.2837 和 81.3108 kWh，架次均值分别为 23.70、26.70、27.40 和 27.00。表中三项指标呈现不同取舍：随机化构造平均架次较少、能耗较低但返航较晚；禁忌搜索平均返航较早，但能耗和架次较高。结果反映的是 30 次有效候选预算下的搜索表现，不构成全局最优性证明。历史 24 架次方案仍作为外部 warm-start 基线单独保留（104.61 min、71.9169 kWh）。

四种方法在返航时间、能耗和架次数上的均值及总体标准差见图 6-1。

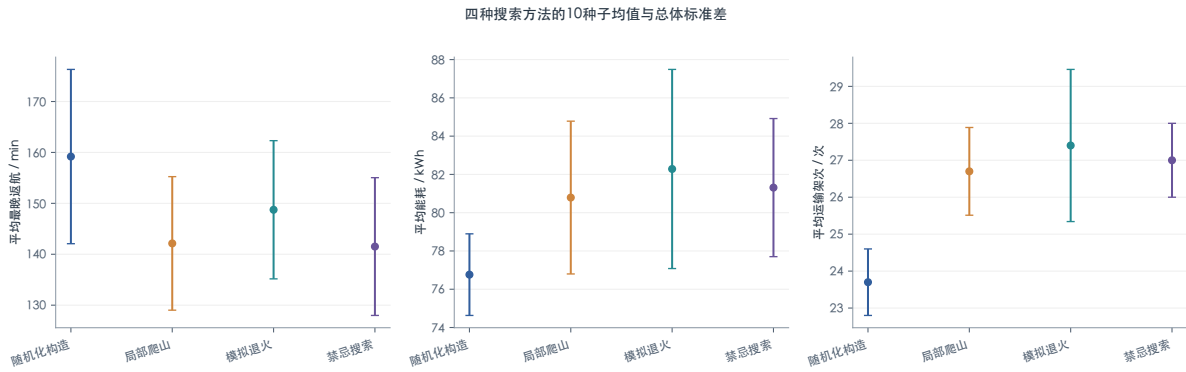


图 6-1 四种搜索方法的 10 种子均值与总体标准差

6.4 当前可执行运输方案

上述历史 warm-start 经统一物理模型、硬时限和资源回放后保留为当前方案。按正式结果文件口径，24 架次完成 80 箱投送，全部运输机于 6276.83 s (104.61 min) 返航，总能耗 71.9169 kWh，优先加权平均交付 45.69 min，最后一箱于 5734.23 s 交付。31 箱硬截止全部满足，非医疗物资加权迟到为 0；最紧硬截止提前 221.03 s，最低返航 SOC 为 20.4065%。8 架实体机和 14 组电池的使用、充电区间无重叠。这是当前执行方案的可行见证，不等同于上表四种方法的独立起点统计结果。

表 6-2 问题二逐箱交付与时限验收汇总

验收对象	箱数	违约/迟到箱	判据
医疗物资	16	0	不晚于附件期望时刻
首批保障货箱	30	0	不晚于首批截止时刻
去重后的硬时限货箱	31	0	兼属两类时取较早截止
非医疗物资期望时刻	64	0	统计软迟到，不作硬否决

表 6-3 问题二各服务区交付完成情况与最紧硬截止余量

服务区	箱数	质量/kg	最后交付/min	硬时限箱	最小提前/s
S001	15	154	20.70	3	2417.73
S002	8	81	55.64	2	551.68
S003	8	81	90.14	2	1819.49
S004	6	59	64.01	2	3359.50
S005	6	59	95.57	2	5065.77
S006	6	59	55.47	2	2657.60
S007	5	45	64.40	2	1177.65
S008	5	45	93.44	2	1593.86
S009	3	25	77.61	2	6715.72
S010	3	25	56.32	2	221.03
S011	3	25	95.47	2	5071.80
S012	3	25	86.01	2	1940.29
S013	3	25	45.14	2	1821.40
S014	3	25	47.99	2	720.57
S015	3	25	47.50	2	5637.13

当前方案的多点航线、运输机排程、电池占用与充电、逐航次能耗、交付时限及返航电量依次见图 6-2-6-7；逐箱交付时间与截止要求见图 6-6及表 6-2。

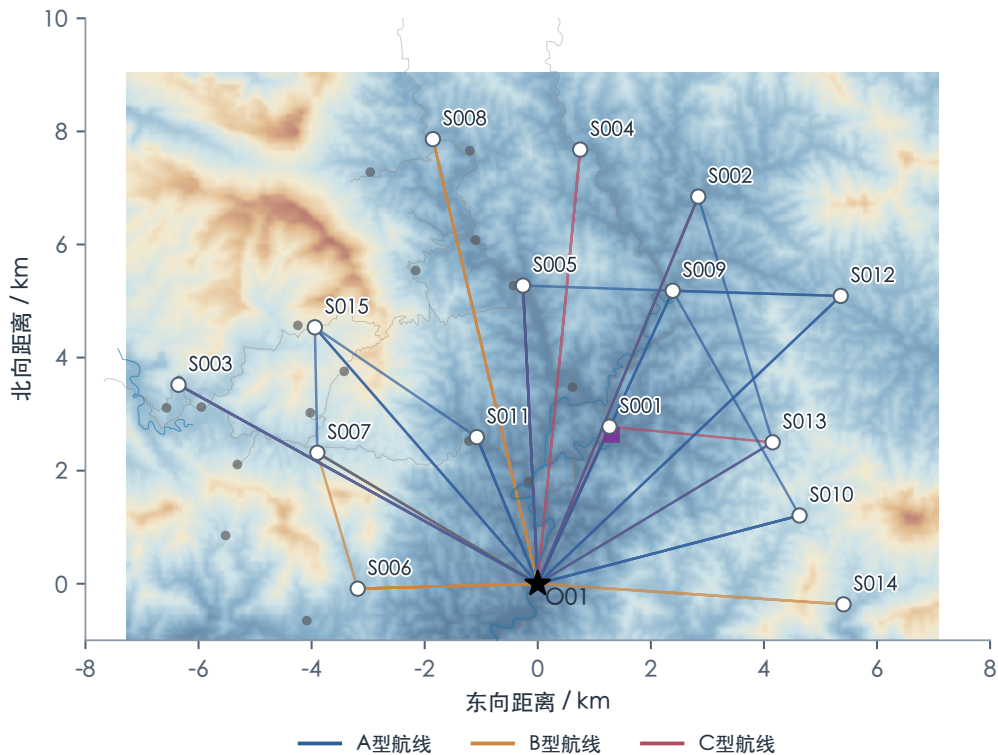


图 6-2 当前 24 架次方案的多点运输航线

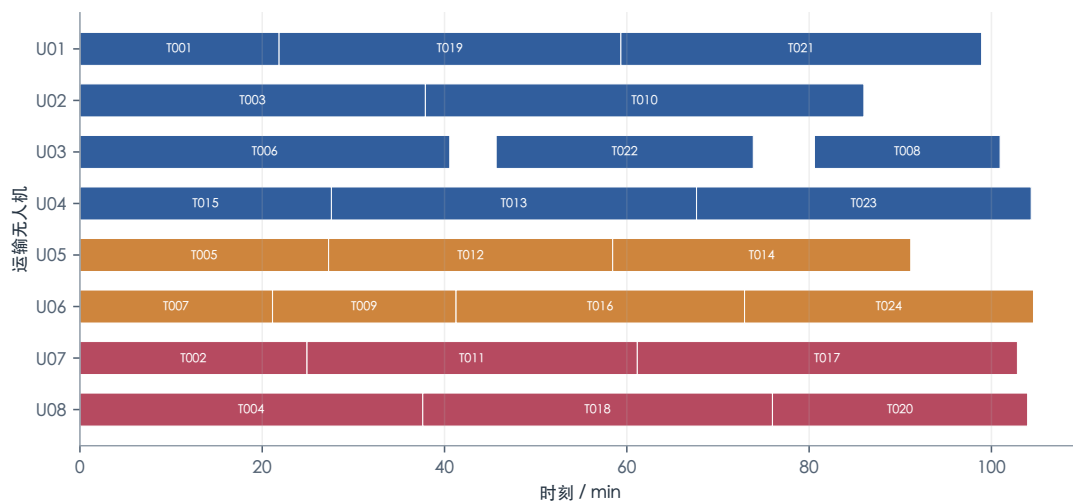


图 6-3 当前方案的运输机排程

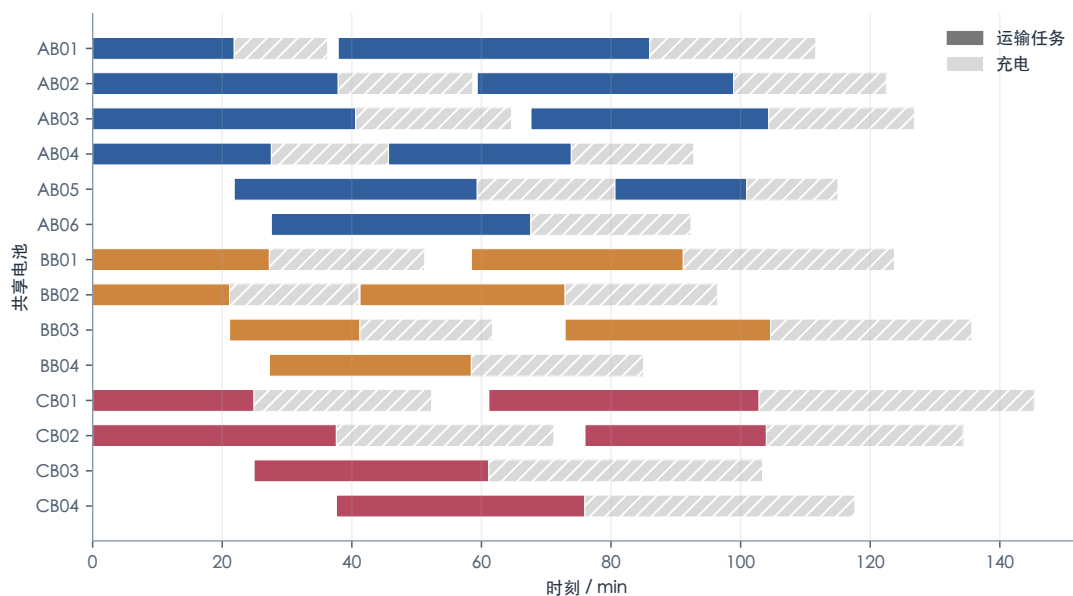
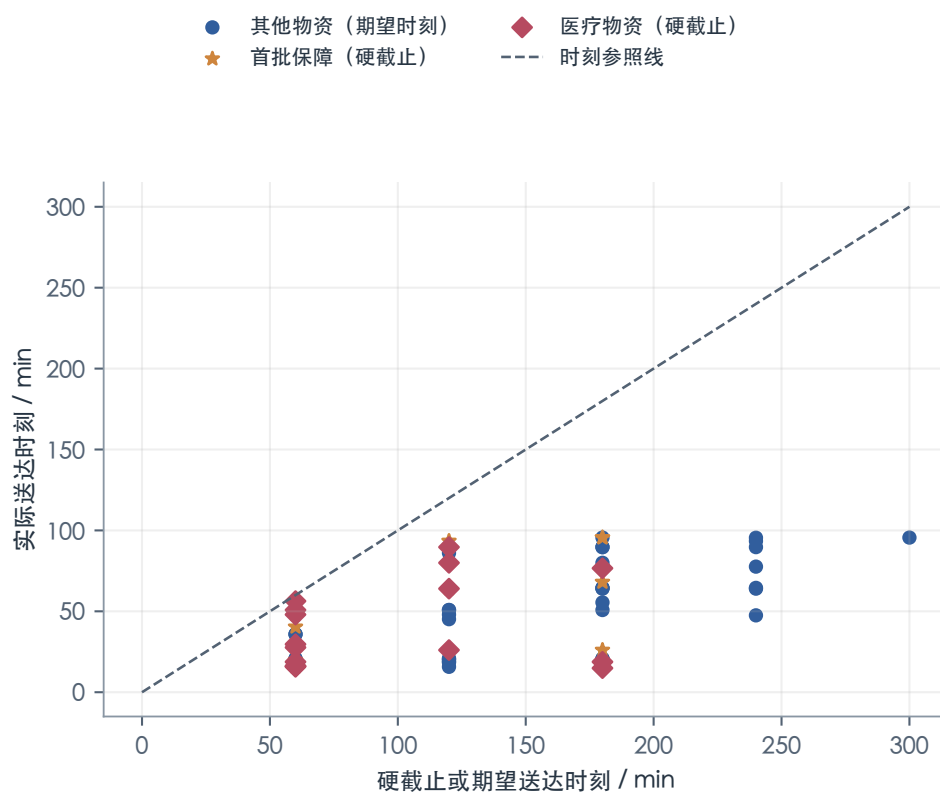
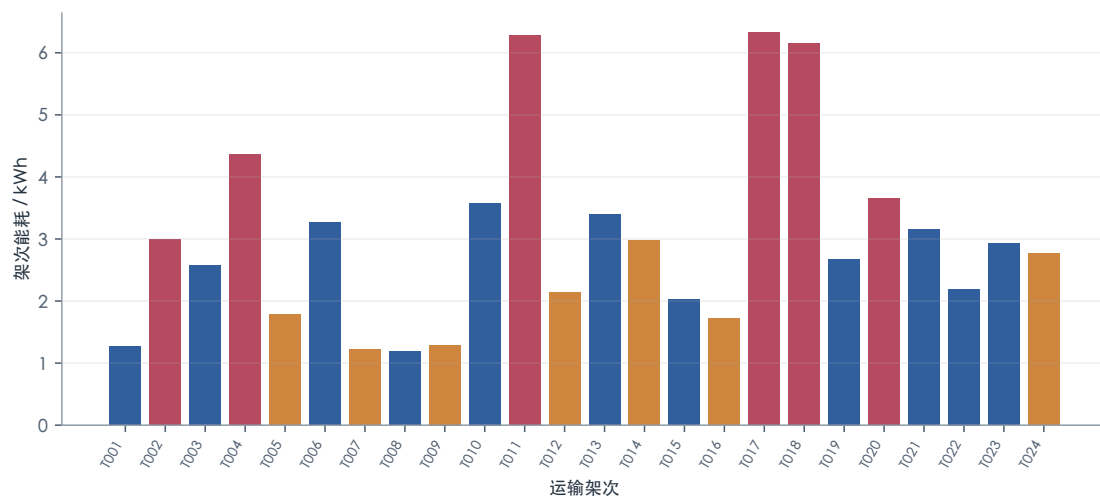


图 6-4 当前方案的运输电池占用与充电区间



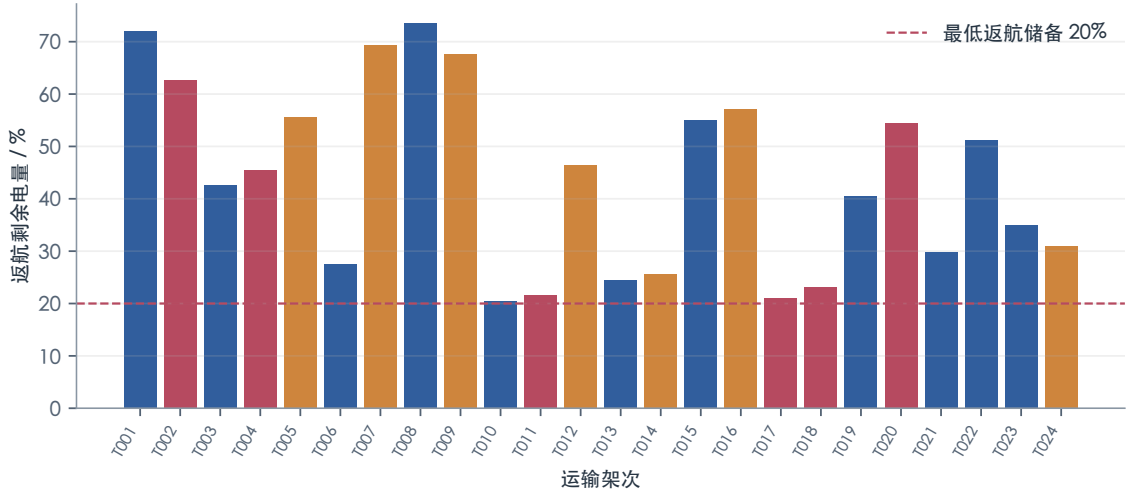


图 6-7 当前方案逐架次返航剩余电量

7. 问题三：连续通信修复与有限候选分解协调

7.1 中继资源与覆盖约束

固定网关 G01 位于 O01 上方 20 m。运输机到网关、运输机到中继、中继到网关的双向允许损耗分别为 122、116、126 dB。对中继任务 ℓ 、候选位置-高度 h 、中继机 u 与能源组件 r ，定义启用和分配变量 $e_\ell, q_{\ell h}, z_{\ell u}^R, w_{\ell r}^R$ ：

$$\sum_h q_{\ell h} = e_\ell, \quad \sum_u z_{\ell u}^R = e_\ell, \quad \sum_r w_{\ell r}^R = e_\ell. \quad (7.1)$$

任务时刻满足 $t_\ell \leq a_\ell \leq b_\ell \leq c_\ell$ 。飞抵、建链、服务、返航及悬停能量等式仅在 $e_\ell = 1$ 时生效； $e_\ell = 0$ 时四个时刻和能耗均固定为 0。中继机占用到返航后 300 s，组件占用延长到充满，并分别施加 **NoOverlap**。

对运输分段 (k, m) 定义直连选择 d_{km} 和由同一中继任务承担接入、回程的 c_{kml} 。连续证书参数为 $\kappa_{km}^{\text{dir}}, \kappa_{kmh}^{\text{acc}}, \kappa_h^{\text{back}}$ ，则

$$d_{km} \leq \kappa_{km}^{\text{dir}}, \quad c_{kml} \leq e_\ell, \quad (7.2)$$

$$c_{kml} \leq \sum_h q_{\ell h} \kappa_{kmh}^{\text{acc}}, \quad c_{kml} \leq \sum_h q_{\ell h} \kappa_h^{\text{back}}, \quad (7.3)$$

$$a_\ell \leq \underline{t}_{km} + M(1 - c_{kml}), \quad b_\ell \geq \bar{t}_{km} - M(1 - c_{kml}), \quad (7.4)$$

$$d_{km} + \sum_\ell c_{kml} \geq 1. \quad (7.5)$$

式 (7.3) 保证接入与回程同时成立，不能拼接两架不同中继。

7.2 连续区间证书命题

链路损耗为

$$L_{ab}(t) = 32.45 + 20 \log_{10}(2400) + 20 \log_{10} D_{ab}(t) + 10 B_{ab}(t). \quad (7.6)$$

命题 (连续覆盖证书)。对运输子区间 I ，若距离上界 $\bar{D}_I \geq \max_{t \in I} D_{ab}(t)$ 且遮挡上界 $\bar{B}_I \geq 10 B_{ab}(t)$ 对 I 内所有时刻成立，令

$$\bar{L}_I = 32.45 + 20 \log_{10}(2400) + 20 \log_{10} \bar{D}_I + \bar{B}_I. \quad (7.7)$$

当直连满足 $\bar{L}_I \leq L_{T \leftrightarrow G}^{\max}$, 或存在同一在役中继使接入与回程两条链路各自的 $\bar{L}_I$ 均不超过相应门限时, I 内每一时刻均双向可通信。

证明。 $20 \log_{10} D$ 关于 $D > 0$ 单调不减, 遮挡项也被 $\bar{B}_I$ 控制, 故任意 $t \in I$ 均有 $L_{ab}(t) \leq \bar{L}_I$ 。中继情形分别对接入和回程应用此不等式, 并由式 (7.3)、(7.4) 保证同一中继全区间在役, 结论成立。 $\square$

程序以爬升、巡航、下降、交接和中继切换为初始分点; 距离上界乘 1.0002 安全因子, 遮挡由 DEM 像元扫掠证明。不能证明时递归二分, 深度用尽即拒绝, 离散采样只作交叉检查。

7.3 有限候选分解协调审计

主问题提出运输族和中继窗口, 通信子问题返回连续证书; 理论上通信冲突应形成覆盖割并回传主问题。当前程序只暴露有限运输族, 没有可调用的通信冲突割接口, 因此本次计算严格称为“有限候选上的通信可行性修复与分解协调审计”, 不称强联合优化或全局最优。

审计三类运输族和 32 个中继窗口候选, 所有候选均重新验证硬时限、资源与连续通信。表 7-1 给出三轮候选规模; 在相同零加权迟到条件下形成 3 个非支配点。本文用时间优先的正式主方案文件作为提交结果, 同时另报最大最小链路裕度候选。连续区间认证核心代码见附录 A.3。

表 7-1 有限候选分解协调审计与连续通信裕度

轮次/汇总	运输族	候选数	获证数	返回裕度/dB
1	hill1	14	14	0.094145
2	newq2	8	8	0.001863
3	q2	10	10	0.001863
裕度最大候选	hill1	32	32	0.094145

主方案共 29 架次 (运输 26、中继 3)。返航时刻为 6769.22 s; 总能耗 78.8732 kWh, 最小链路裕度 0.093780 dB。

最大裕度候选采用东部窗口 3500 s, 返航 6769.31 s、能耗 79.0401 kWh, 29 个总架次, 最小裕度 0.094145 dB。其裕度只增加 0.000365 dB, 不能视为实质稳健化; 两种方案在现场执行前都须重新测量链路与地形。

主方案 80 箱均在原服务区交接, 31 箱硬截止和 64 箱非医疗物资期望时刻均无违约; 最后一箱 6317.40 s 交接。运输机 6625.61 s 收尾, 运输与中继分别耗电 75.3988 和 3.4744 kWh。最紧硬截止提前 84.65 s, 最低运输机与中继返航 SOC 分别为 20.4065% 和 38.6085%。4061 个连续区间全部获证, 最窄区间的保守裕度为 0.093780 dB。三处悬停离地高度都等于 300 m 上限, 也没有高度扰动余量。

为比较通信约束加入前后的空间方案, 图 7-1 在同一 DEM 上以问题二运输方案为基线, 展示问题三联合航线、分机型航线及中继位置; 其中 (a) 为问题二方案, (b)–(f) 为问题三方案。中继点及其对直连盲区的覆盖关系另见图 7-2 和图 7-3; 任务时间关系与逐段通信提供方见图 7-4、图 7-5 和图 7-6。链路裕度分布与中继返航电量分别见图 7-9 和图 7-8。

表 7-2 问题三三班中继任务的悬停位置与资源

任务	点	机体/组件	经纬度	海拔/m
RP01	W	R01/RB01	109.2016, 23.0424	774.3
RP02	E	R02/RB02	109.2821, 23.0288	674.9
RP03	N	R02/RB03	109.2309, 23.0627	533.2

表 7-3 问题三三班中继任务的时窗与能源（续）

任务	建链/min	服务结束/min	返航/min	能耗/kWh	返航 SOC/%
RP01	11.53	103.89	112.82	1.965	38.61
RP02	12.13	54.40	63.78	1.076	66.37
RP03	91.24	98.50	107.42	0.434	86.44

表 7-4 问题三连续通信区间证书按连接方式汇总

连接方式	对应链路	已验证区间数	最小证明裕量/dB
G01	固定网关直连	2229	0.09378
R01	经西部中继机	971	0.09415
R02	经东部或北部中继机	861	0.18628
合计	连续通信证书	4061	0.09378

表 7-5 问题二运输方案与问题三通信约束下联合方案对比

指标	问题二	问题三	差值
运输架次	24	26	+2
中继架次	0	3	+3
全部设备返航/min	104.61	112.82	+8.21
总能耗/kWh	71.9169	78.8732	+6.9563
最后一箱交付/min	95.57	105.29	+9.72
普通物资加权迟到/系数 ·s	0	0	0

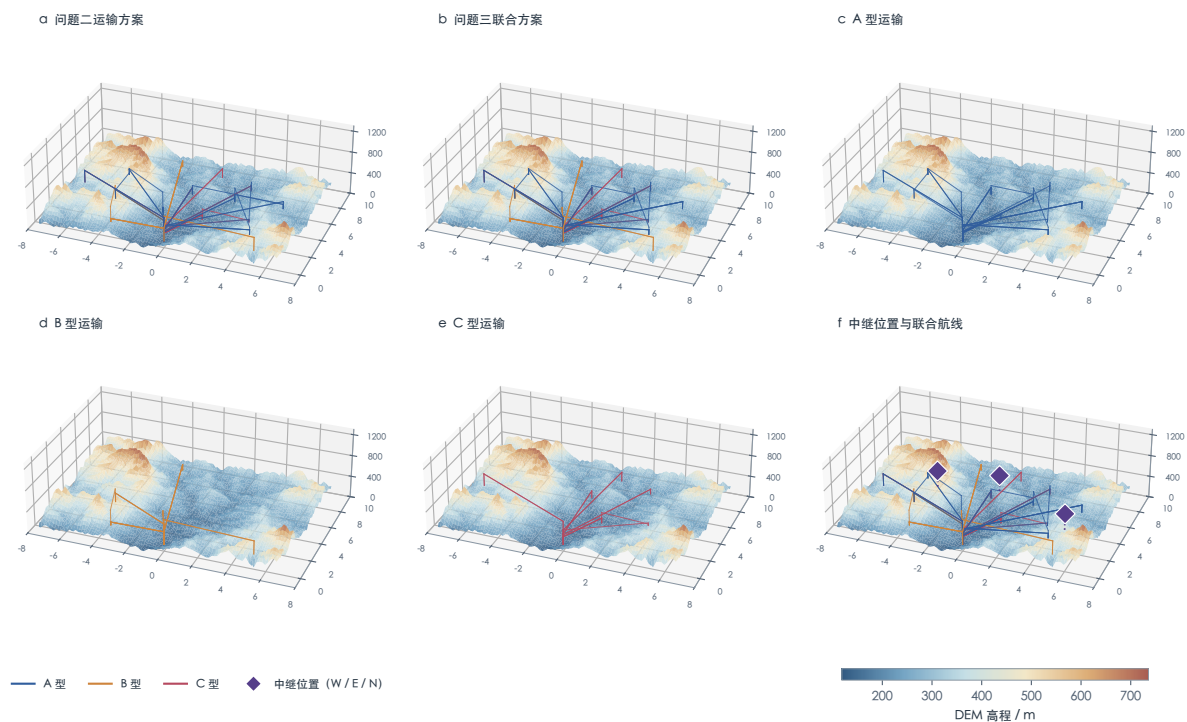


图 7-1 问题二与问题三运输航线、机型分布及中继位置的 DEM 三维对比

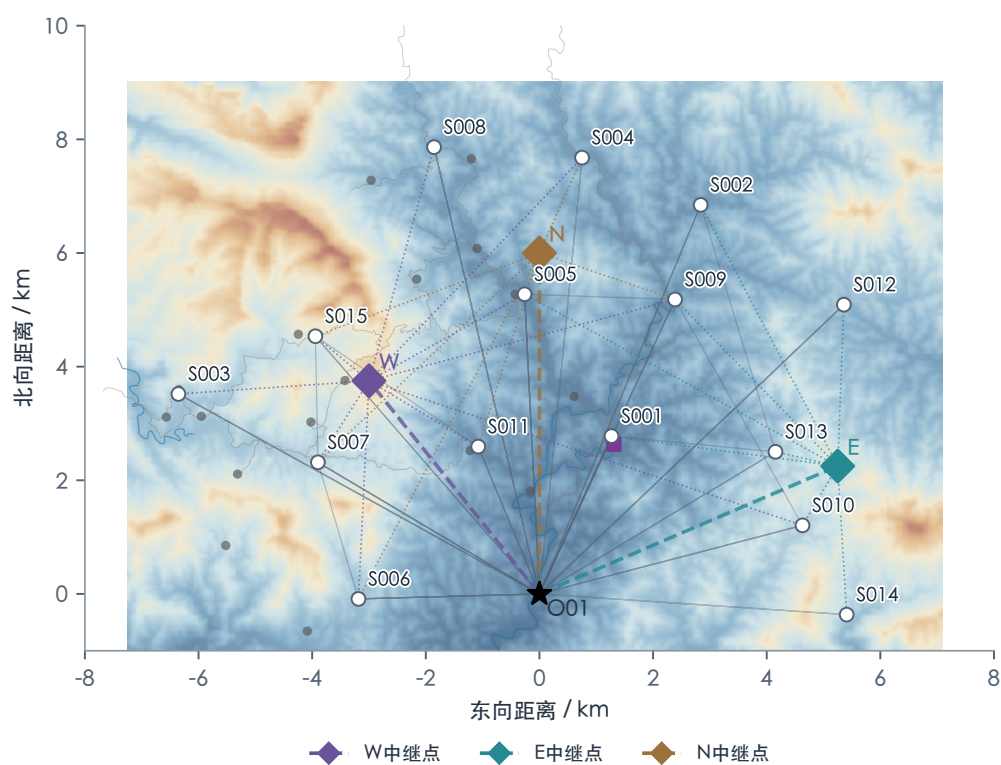


图 7-2 主方案西、东、北中继悬停点

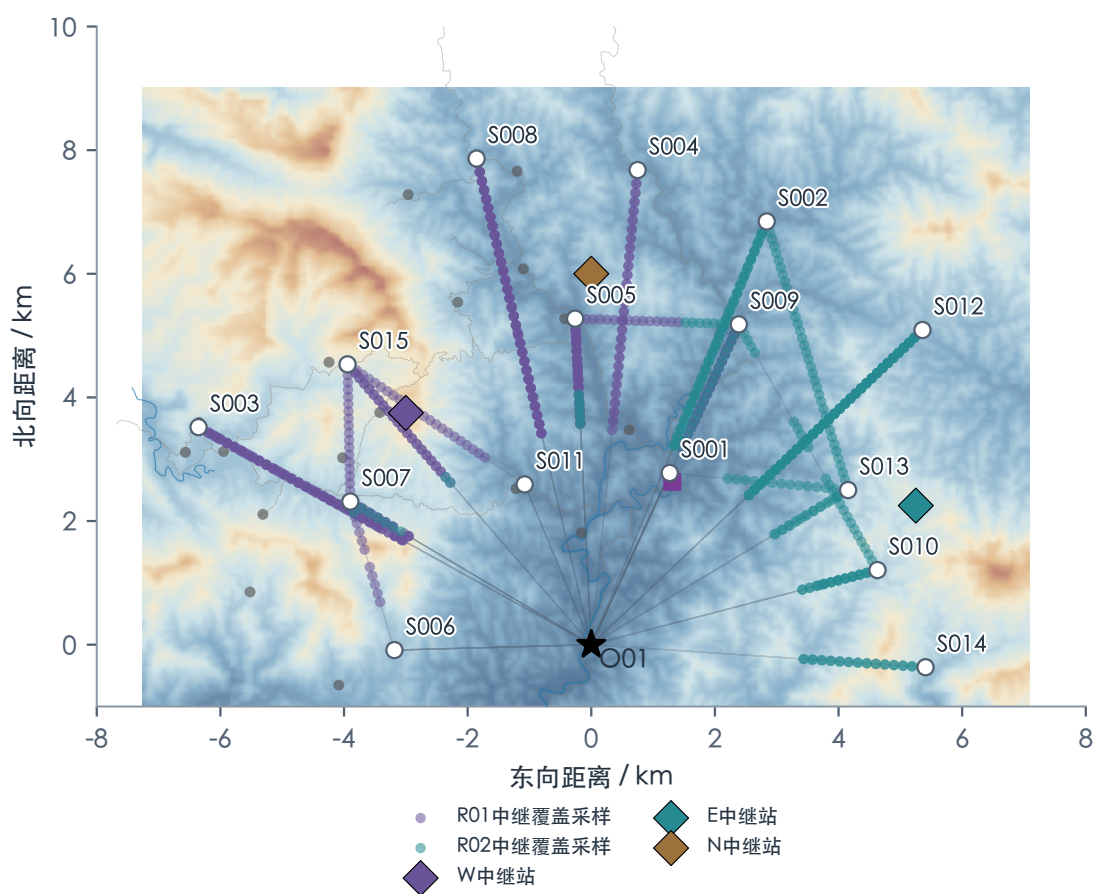


图 7-3 运输盲区采样点与中继覆盖关系

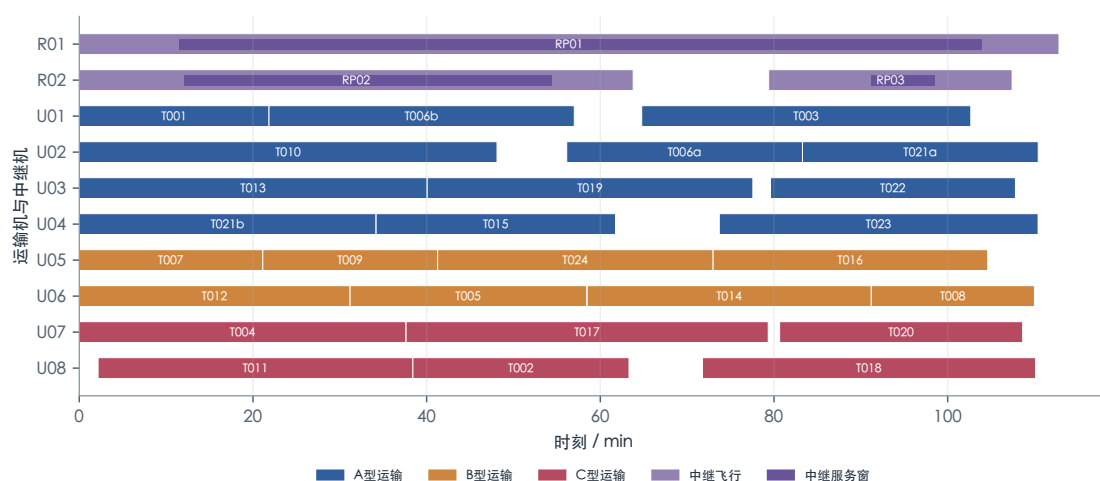


图 7-4 运输与中继联合时间表



图 7-5 运输任务各时段的通信提供方

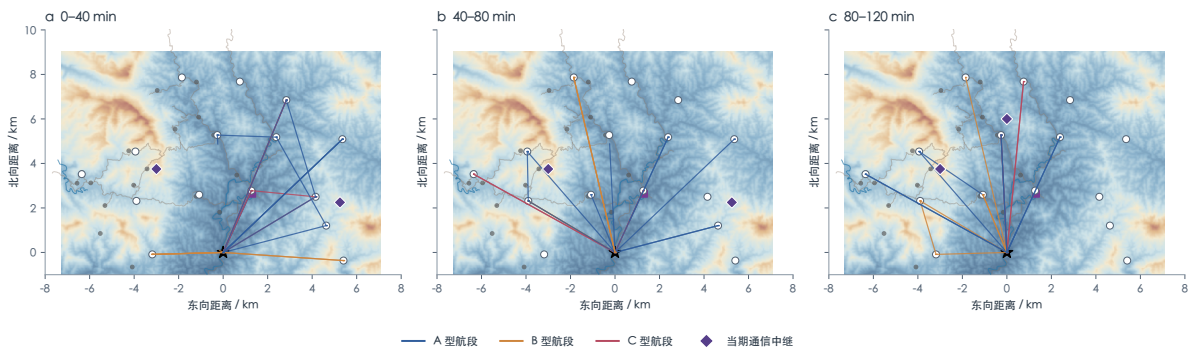


图 7-6 不同时间段的运输航线与中继空间分布

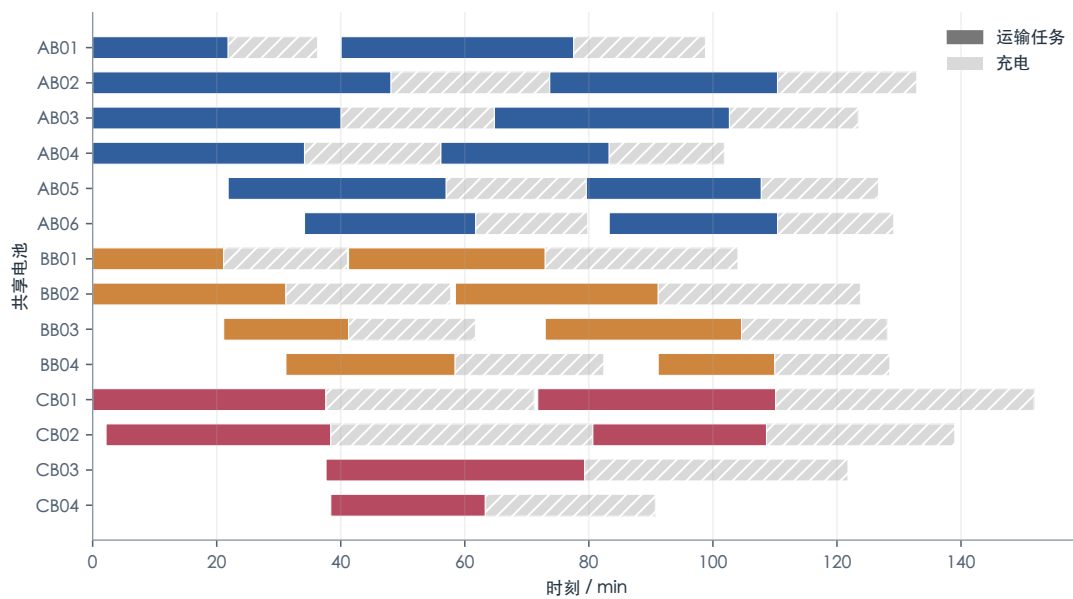


图 7-7 运输电池占用与充电区间

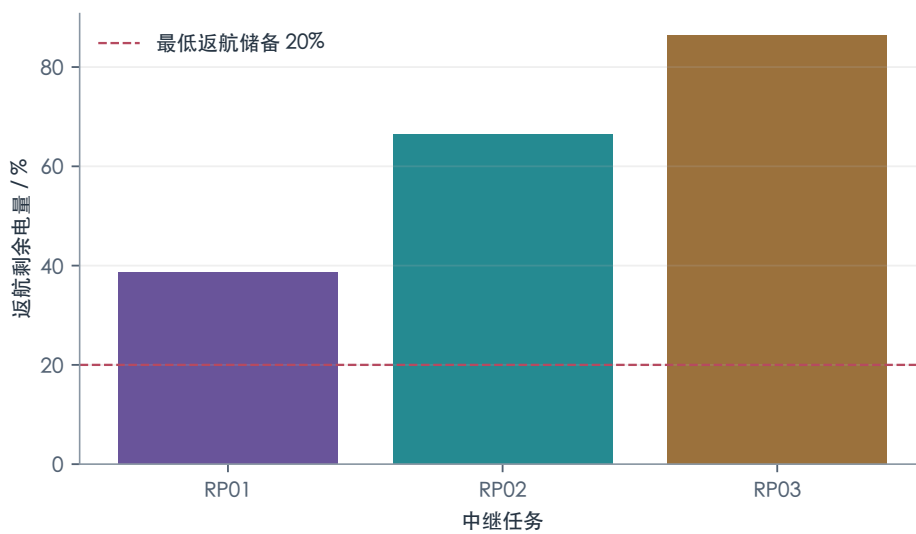


图 7-8 三项中继任务的返航剩余电量

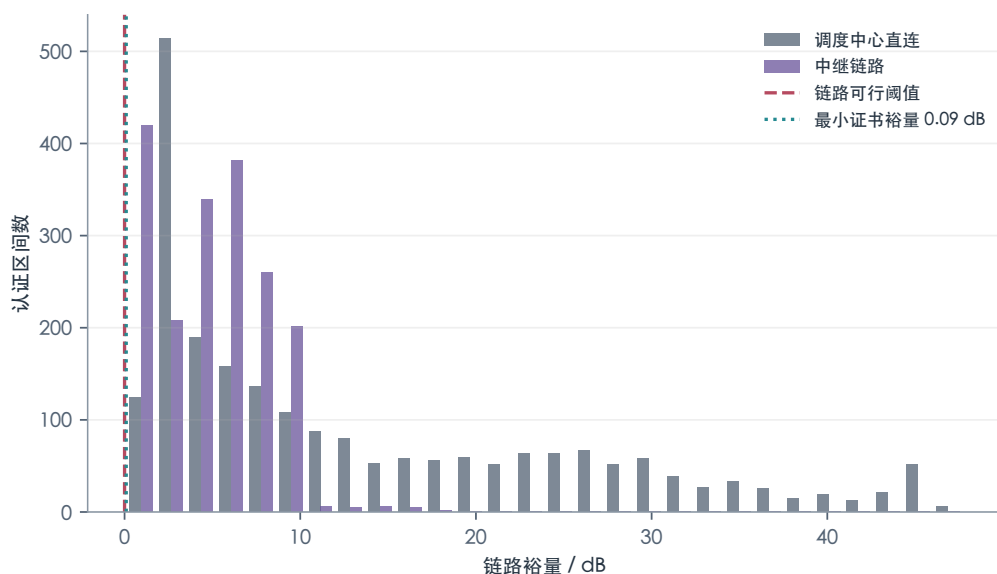


图 7-9 连续通信区间的保守链路裕度分布

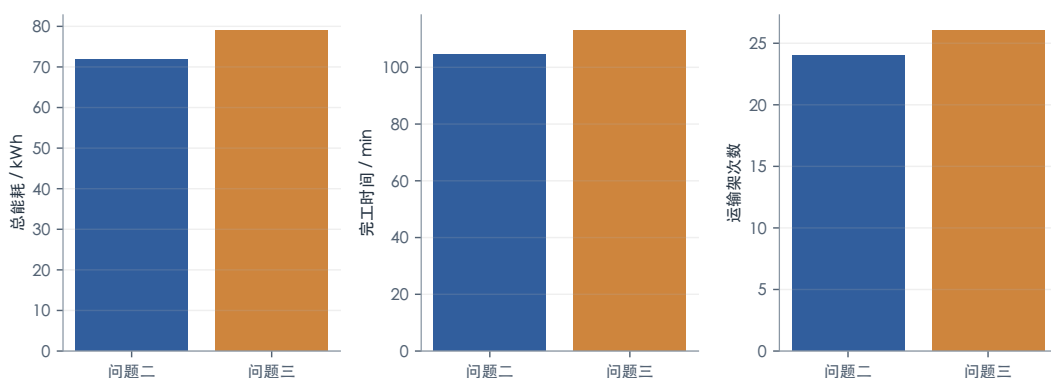


图 7-10 问题二与问题三运输和通信方案指标比较

8. 问题四：固定联合任务的独立分区

问题四建立在问题三联合运输-通信方案的基础上。与前三问不同，本问不再重新设计运输路线或中继调度，而是研究：当已经确定的联合任务被划分为若干相互独立的执行单元后，原本能够在统一资源池中共享的运输无人机、电池和中继资源将发生怎样的配置变化。

因此，本文固定问题三主方案中的货箱组批、服务区访问顺序、运输机型、中继任务及其起止时刻，仅将服务区所属任务组作为新的决策对象。这一处理使问题四与问题三之间保持严格的数据继承关系，同时避免将“任务分区”与“重新优化调度”混为一谈。

8.1 不可拆任务组件与资源配置模型

问题四建立在问题三联合运输-通信方案的基础上。与前三问继续优化运输或通信方案不同，本问要求在问题三任务结构保持不变的条件下，将全部服务区划分为若干能够独立执行的任务组，并分析任务组之间失去资源共享后所产生的额外配置需求。因此，本问的核心不再是重新设计运输路线，而是研究固定联合任务在不同组织分区下的资源峰值及独立执行能力。

具体而言，本文固定问题三主方案中的货箱组批方式、服务区访问顺序、运输机型、运输任务起止时刻、中继任务起止时刻以及既定通信保障关系。在此基础上，仅将各服务区所属的任务组作为新的决策变量。这种处理既保证问题四与问题三之间的任务结构完全衔接，又能够单独刻画“统一资源池被拆分”后产生的资源配置代价。

8.1.1 不可拆任务组件的构造

问题三中部分运输架次会连续访问多个服务区。若同一运输架次依次访问服务区 S_i 与 S_j ，则在问题四中二者不能被分配至不同任务组，否则同一条已经固定的运输任务将被人为拆分，与题目要求保持问题三任务结构不变相矛盾。

为形式化描述这一约束，构造服务区关联图

$$G_S = (V_S, E_S), \quad (8.1)$$

其中节点集合为

$$V_S = \{S001, S002, \dots, S015\}. \quad (8.2)$$

若问题三中存在至少一个运输架次同时访问服务区 S_i 与 S_j ，则在二者之间加入无向边

$$(S_i, S_j) \in E_S. \quad (8.3)$$

由于同一路线中的服务区必须属于同一任务组，因此图 G_S 的每一个连通分量均构成不可继续拆分的最小任务单元。依据问题三确定的 26 条运输航次，最终得到如下 8 个不可拆任务组件：

$$\begin{aligned} C_1 &= \{S001, S002, S013\}, \\ C_2 &= \{S003\}, \\ C_3 &= \{S004\}, \\ C_4 &= \{S005, S009, S010\}, \\ C_5 &= \{S006, S007, S011, S015\}, \\ C_6 &= \{S008\}, \\ C_7 &= \{S012\}, \\ C_8 &= \{S014\}. \end{aligned} \quad (8.4)$$

因此，问题四的组合决策对象由 15 个服务区进一步压缩为 8 个不可拆组件。这一转换没有改变问题三中的任何运输任务，只是利用既有路线结构提前消除了不满足题意的分组方式，从而将后续任务分区转化为对集合

$$\mathcal{C} = \{C_1, C_2, \dots, C_8\} \quad (8.5)$$

的集合划分问题。

服务区之间由问题三多点运输架次建立的关联关系如图 8-1 所示；连通分量整体进入后续分区枚举。

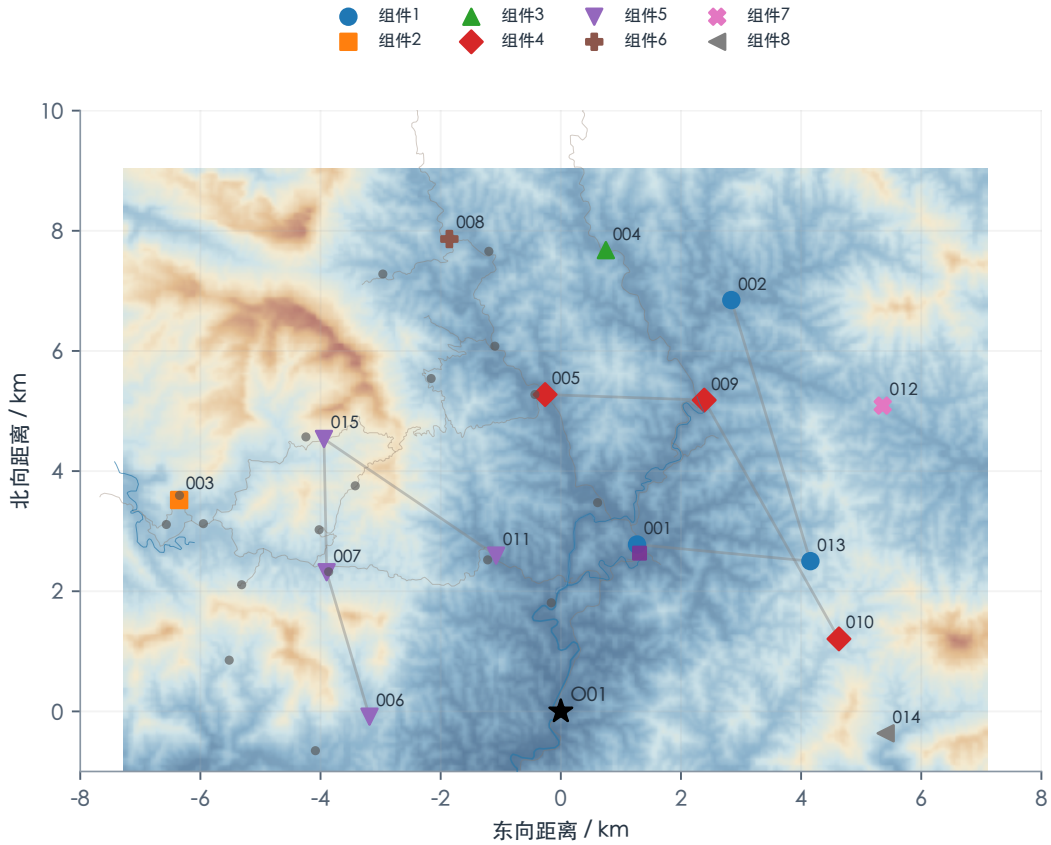


图 8-1 固定运输任务诱导的服务区关联网络

8.1.2 分区空间及完全枚举

设需要将 8 个不可拆组件划分为 H 个非空任务组。由于任务组本身不存在固定编号，交换两个组的编号不会产生新的实际任务划分。因此，8 个组件划分为 H 个非空无标签子集的方案数量等于第二类 Stirling 数

$$S(8, H) = \frac{1}{H!} \sum_{j=0}^H (-1)^j \binom{H}{j} (H-j)^8. \quad (8.6)$$

当划分为两组时，

$$S(8, 2) = \frac{1}{2!} (2^8 - 2) = 127. \quad (8.7)$$

当划分为三组时，

$$S(8, 3) = \frac{1}{3!} (3^8 - 3 \times 2^8 + 3) = 966. \quad (8.8)$$

因此，两组和三组情形下分别仅需考察 127 种和 966 种不同划分。在这一规模下，可以对全部任务分区进行完全枚举，无需借助随机搜索或启发式算法。需要说明的是，这里的“完全枚举”仅针对问题三任务完全固定后的分区决策，并不意味着前述运输-通信联合调度本身获得了全局最优解。分区枚举与资源评价核心代码见附录 A.4。

8.1.3 固定任务下的资源峰值计算

任务分区以后，各任务组被要求具备独立执行能力，即不能在执行过程中临时借用其他组的无人机、电池或中继资源。因此，一组任务所需的最低资源数量不能通过简单累计任务次数获得，而应由固定时间轴上的最大并发占用决定。

设第 h 个任务组中使用运输机型 g 的航次集合为 $\mathcal{R}_{hg}$ 。对于任一运输航次 $r \in \mathcal{R}_{hg}$ ，记其开始时刻和返航结束时刻分别为 s_r 与 C_r 。则时刻 t 对应的同型运输无人机并发占用量为

$$N_{hg}^U(t) = \sum_{r \in \mathcal{R}_{hg}} \mathbf{1}_{[s_r, C_r)}(t), \quad (8.9)$$

其中 $\mathbf{1}_I(t)$ 为区间 I 的示性函数。于是第 h 组对机型 g 的最低实体无人机需求为

$$r_{hg}^U = \max_t N_{hg}^U(t). \quad (8.10)$$

对于共享电池，资源占用不能在运输机返航时立即结束。电池在完成一个航次后必须继续充电至 100% SOC，在充电结束之前均不能再次投入任务。若航次 r 对应电池的充电完成时刻为 C_r^B ，则电池占用函数为

$$N_{hg}^B(t) = \sum_{r \in \mathcal{R}_{hg}} \mathbf{1}_{[s_r, C_r^B)}(t), \quad (8.11)$$

从而该组对相应型号电池的最低需求为

$$r_{hg}^B = \max_t N_{hg}^B(t). \quad (8.12)$$

中继资源采用相同的最大并发思想。若中继任务集合记为 $\mathcal{L}_h$ ，则中继无人机在任务结束返航后仍需计入 300 s 周转时间。因此，中继无人机最低配置量为

$$r_h^R = \max_t \sum_{\ell \in \mathcal{L}_h} \mathbf{1}_{[s_\ell, C_\ell + 300)}(t). \quad (8.13)$$

中继能源组件则从任务开始一直占用到其完成充电，其最低需求量同样由相应占用区间的最大重叠数确定。

上述资源占用函数均为分段常数函数，只有在任务开始、任务结束、周转结束以及充电结束等事件时刻才会发生变化。因此，固定任务下的资源峰值可以通过有限事件时刻的扫描精确获得，无需对整个连续时间轴进行逐秒离散。这一处理能够同时计入任务执行与任务后周转，避免仅依据航次数量估计资源配置而低估电池或中继组件需求。

需要特别说明，若问题三中的某一中继任务同时服务多个运输任务，而这些运输任务在问题四中被划入不同任务组，则在“完全独立执行”的要求下，各任务组均必须分别具备相应的中继保障能力，不能继续跨组共享原问题三中的同一中继资源。因此，本问计算的是固定问题三计划被拆分之后的独立资源峰值，而不是重新优化运输和中继任务。

8.1.4 重复配置、库存闲置与资源缺口

为评价任务分区对资源利用效率的影响，本文将资源配置结果分解为“重复配置”、“库存闲置”和“库存缺口”三类指标。

资源类型按顺序记为 A、B、C 型运输无人机，A、B、C 型共享电池，中继无人机以及中继能源组件。现有库存向量为

$$\mathbf{A} = (4, 2, 2, 6, 4, 4, 2, 6). \quad (8.14)$$

设任务组 h 对第 k 类资源的最低配置需求为 r_{hk} ，则所有任务组完全独立执行时，该类资源的总配置需求为

$$R_k = \sum_h r_{hk}. \quad (8.15)$$

进一步记不进行任务分区、仍采用统一资源池执行问题三固定任务时，第 k 类资源的峰值需求为 r_k^{pool} ，现有库存量为 A_k 。

定义重复配置量为

$$R_k^{\text{dup}} = R_k - r_k^{\text{pool}}, \quad (8.16)$$

它衡量任务分区后相对于统一资源池额外增加的资源数量。

定义库存闲置量为

$$I_k^{\text{idle}} = \max(0, A_k - R_k), \quad (8.17)$$

表示完成当前独立分区最低配置后，现有库存中仍未被使用的资源数量。

定义资源缺口为

$$G_k^{\text{short}} = \max(0, R_k - A_k), \quad (8.18)$$

表示在现有库存之外，为了保证所有任务组完全独立执行仍需补充的资源数量。

三类指标具有不同的物理含义。其中，重复配置反映资源共享被切断后造成的额外需求；库存闲置描述某一类现有资源是否存在剩余；资源缺口则直接反映当前库存能否支撑任务组完全独立运行。由于不同型号运输机、电池和中继资源之间不可任意替代，因此不能用某类资源的库存闲置去抵消另一类资源的配置缺口。

8.1.5 词典序分区目标

在任务分区决策中，本文首先关注方案能否尽可能接近现有库存条件下的独立可执行状态，其次考虑总体资源配置规模，最后再比较任务组之间的工作量均衡程度。

设第 h 个任务组的固定运输任务工作量为 W_h ，则采用如下词典序目标：

$$\min_{\text{lex}} \left(\sum_k G_k^{\text{short}}, \sum_k R_k, \text{CV}(W_1, \dots, W_H) \right). \quad (8.19)$$

其中工作量变异系数为

$$\text{CV}(W_1, \dots, W_H) = \frac{\sigma_W}{\bar{W}}, \quad (8.20)$$

$\bar{W}$ 与 σ_W 分别表示各任务组工作量的均值和标准差。

该目标的第一层优先最小化库存缺口，因为资源缺口直接决定当前库存距离完全独立执行还有多大差距；第二层在缺口相同的方案中减少总配置数量，抑制因资源池拆分而产生的不必要重复配置；第三层再尽可能改善各组固定任务量之间的均衡程度。

因此，本问最终选择的分区首先是“资源可实施性优先”意义下的方案，并不保证工作量变异系数单独达到最小。这一目标优先级也将在后续结果中解释为何资源缺口较小的方案仍可能表现出较明显的组间工作量不均衡。

8.2 两组与三组分区结果

基于上一节得到的 8 个不可拆任务组件，分别对 127 种两组划分和 966 种三组划分进行完全枚举，并按照“总库存缺口-总配置规模-工作量变异系数”的词典序目标进行比较。最终得到的两组和三组代表方案如表 8-1 和图 8-2 所示。

两组情况下，最优分区将 S014 单独划为一个任务组，其余 14 个服务区组成另一任务组。其中主任务组承担 77 箱货物和 25 个运输架次，固定运输任务工作量为 13.19 h；S014 单独组成的任务组承担 3 箱货物和 1 个运输架次，固定任务工作量为 0.52 h。

三组情况下，S003 和 S012 分别单独形成任务组，其余 13 个服务区组成主任务组。三个任务组分别承担 69 箱、8 箱和 3 箱货物，对应运输架次数分别为 22 次、2 次和 2 次，固定任务工作量分别为 11.25 h、1.31 h 和 1.16 h。

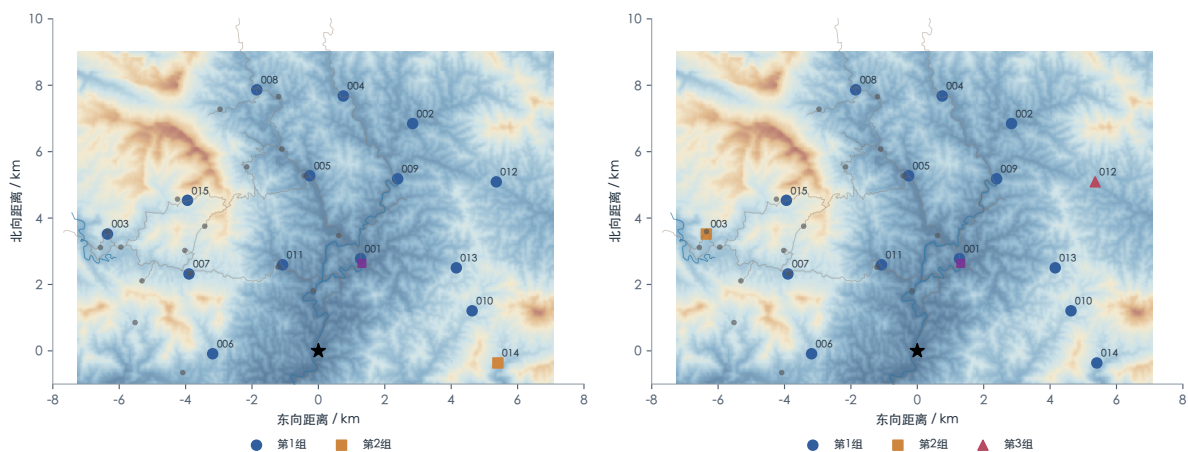


图 8-2 固定问题三计划下的两组与三组独立任务分区

图 8-3 在地形背景上补充了参考飞行时间与两种任务分区，图 8-4 则概括各组服务点的空间分布范围。

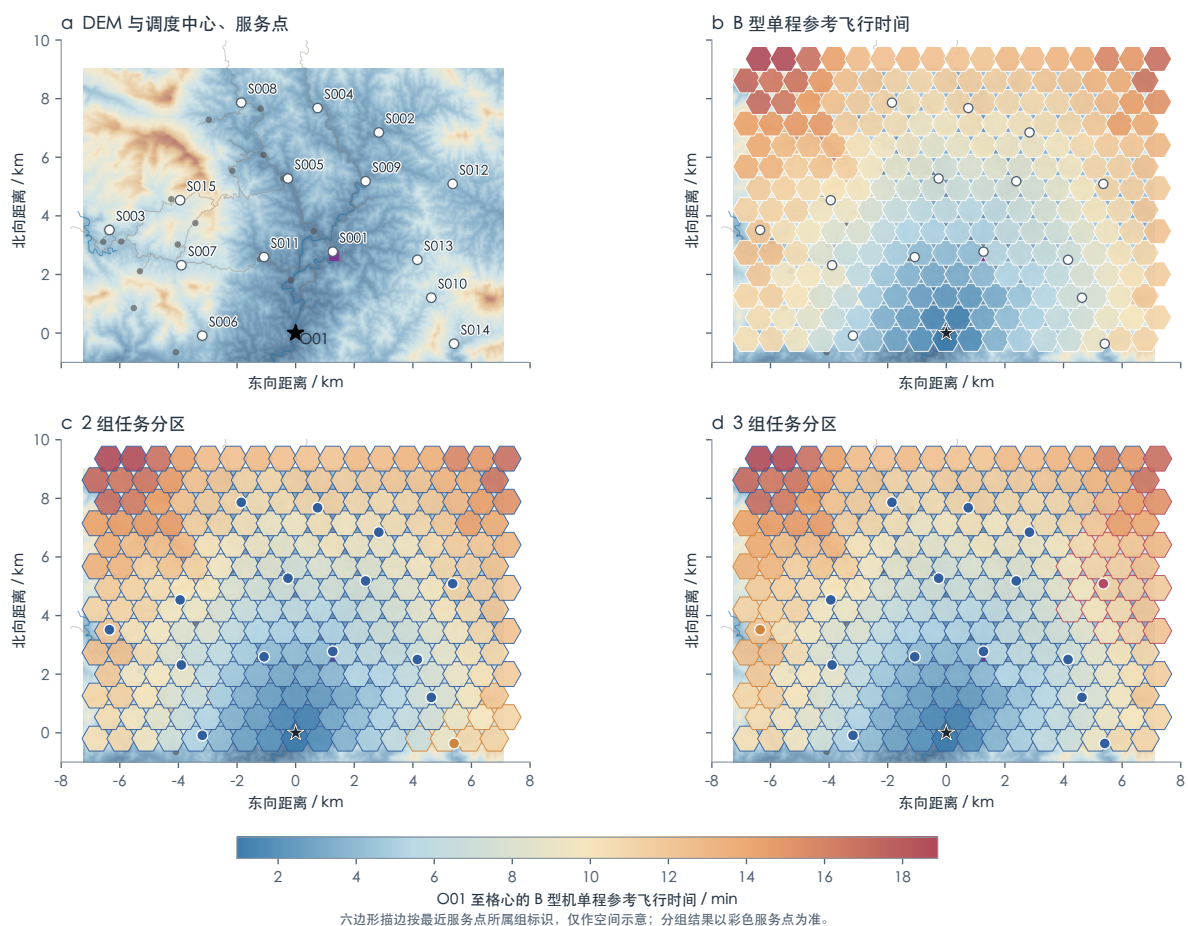


图 8-3 DEM 地形、参考飞行时间与两组及三组任务分区

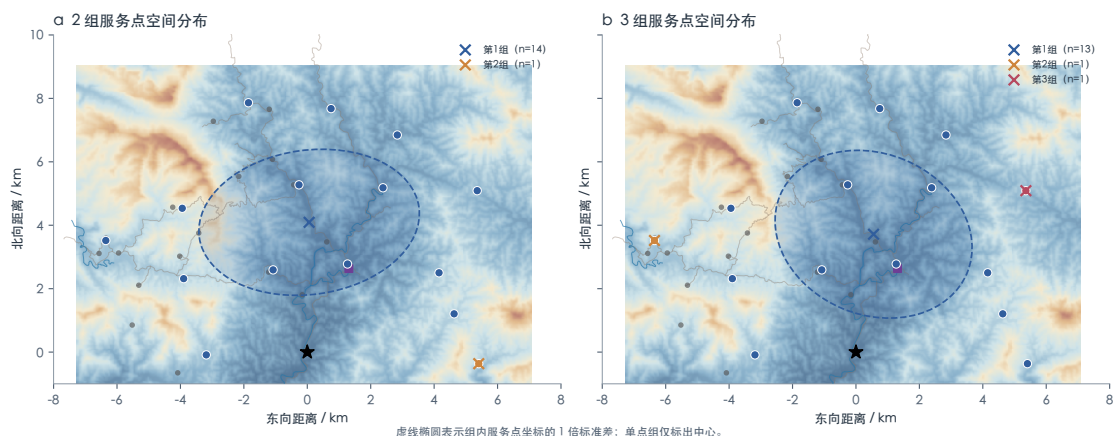


图 8-4 两组分区的服务点空间分布与标准差椭圆

表 8-1 问题四两组和三组分区的服务区、货箱与固定任务工作量

分组	服务区	箱数	运输架次	工作量/h
2 组-1	S001, S002, S003, S004, S005, S006, S007, S008, S009, S010, S011, S012, S013, S015	77	25	13.19
2 组-2	S014	3	1	0.52
3 组-1	S001, S002, S004, S005, S006, S007, S008, S009, S010, S011, S013, S014, S015	69	22	11.25
3 组-2	S003	8	2	1.31
3 组-3	S012	3	2	1.16

从空间分布可以看出，最终分区并不是按照地理位置简单地将服务区平均切割。其根本原因在于问题四必须继承问题三已经形成的多点运输任务。一旦若干服务区位于同一不可拆连通组件中，便只能整体进入同一任务组。因此，问题四的分区结果受到既有运输结构和资源并发关系的共同约束，空间邻近性并不是唯一决定因素。

8.2.1 两组方案的资源配置结果

根据固定任务的最大并发资源需求，两组方案中包含其余 14 个服务区的主任务组所需资源向量为

$$(4, 2, 2, 6, 4, 4, 2, 3), \quad (8.21)$$

而 S014 单独任务组的资源需求向量为

$$(0, 1, 0, 0, 1, 0, 1, 1). \quad (8.22)$$

因此，两组完全独立执行时的总资源配置向量为

$$\mathbf{R}^{(2)} = (4, 3, 2, 6, 5, 4, 3, 4). \quad (8.23)$$

与现有库存

$$\mathbf{A} = (4, 2, 2, 6, 4, 4, 2, 6) \quad (8.24)$$

比较可得，两组方案的资源缺口向量为

$$\mathbf{G}^{(2)} = (0, 1, 0, 0, 1, 0, 1, 0). \quad (8.25)$$

因此，在保持问题三固定任务不变且要求两个任务组完全独立执行时，现有库存至少还缺少 1 架 B 型运输无人机、1 组 B 型共享电池以及 1 架中继无人机，总资源缺口为 3 件。

值得注意的是，两组方案同时存在 2 组闲置的中继能源组件。这并不与“仍缺 3 件资源”矛盾。原因在于不同类型资源之间不能相互替代：闲置的中继能源组件无法承担 B 型运输机、B 型电池或中继无人机的功能。因此，在独立任务配置分析中，必须分别比较各类资源供需，不能只使用资源总件数判断方案是否可行。

8.2.2 三组方案的资源配置结果

三组情况下，由其余 13 个服务区构成的主任务组资源需求为

$$(3, 2, 2, 5, 4, 3, 2, 3), \quad (8.26)$$

S003 单独任务组的资源需求为

$$(1, 0, 1, 1, 0, 1, 1, 1), \quad (8.27)$$

S012 单独任务组的资源需求为

$$(2, 0, 0, 2, 0, 0, 1, 1). \quad (8.28)$$

三组完全独立执行时，总资源配置向量为

$$\mathbf{R}^{(3)} = (6, 2, 3, 8, 4, 4, 4, 5). \quad (8.29)$$

与现有库存比较后，对应资源缺口向量为

$$\mathbf{G}^{(3)} = (2, 0, 1, 2, 0, 0, 2, 0). \quad (8.30)$$

因此，三组方案至少还需增加 2 架 A 型运输无人机、1 架 C 型运输无人机、2 组 A 型共享电池以及 2 架中继无人机，总资源缺口达到 7 件。

表 8-2 独立分区的固定任务资源向量与库存缺口

方案或分组	箱数	$(A, B, C, B_A, B_B, B_C, R, B_R)$
库存	—	$(4, 2, 2, 6, 4, 4, 2, 6)$
两组：其余 14 区	77	$(4, 2, 2, 6, 4, 4, 2, 3)$
两组：S014	3	$(0, 1, 0, 0, 1, 0, 1, 1)$
两组合计	80	$(4, 3, 2, 6, 5, 4, 3, 4)$
两组缺口	—	$(0, 1, 0, 0, 1, 0, 1, 0)$
三组：其余 13 区	69	$(3, 2, 2, 5, 4, 3, 2, 3)$
三组：S003	8	$(1, 0, 1, 1, 0, 1, 1, 1)$
三组：S012	3	$(2, 0, 0, 2, 0, 0, 1, 1)$
三组合计	80	$(6, 2, 3, 8, 4, 4, 4, 5)$
三组缺口	—	$(2, 0, 1, 2, 0, 0, 2, 0)$

与两组方案相比，三组方案的资源缺口由 3 件增加至 7 件。这一变化并非由运输货物总量增加所致，因为两种分组均执行完全相同的问题三固定任务；其本质原因是任务组数量增加以后，统一资源池原有的跨任务错峰复用能力受到进一步削弱。

例如，一架无人机在统一调度中可以先后执行属于不同服务区的多个任务，但在完全独立分区后，若这些任务分别属于不同任务组，则每个任务组必须分别具备相应的实体机能力。共享电池和中继无人机也存在同样的现象。因此，独立分区会将原本在时间轴上可以共享的资源峰值转化为不同任务组内部的独立峰值，进而产生额外资源配置。

8.2.3 重复配置、库存闲置与库存缺口比较

为进一步解释资源缺口的来源，表 8-3 分别给出了两组和三组方案的重复配置、库存闲置与库存缺口。

两组方案的重复配置向量为

$$(0, 1, 0, 0, 1, 0, 1, 1), \quad (8.31)$$

重复配置总量为 4 件；库存闲置向量为

$$(0, 0, 0, 0, 0, 0, 0, 2), \quad (8.32)$$

即仅有 2 组中继能源组件未被最低配置使用；资源缺口向量为

$$(0, 1, 0, 0, 1, 0, 1, 0), \quad (8.33)$$

总缺口为 3 件。

三组方案的重复配置向量增加为

$$(2, 0, 1, 2, 0, 0, 2, 2), \quad (8.34)$$

总重复配置达到 9 件；库存闲置向量为

$$(0, 0, 0, 0, 0, 0, 0, 1), \quad (8.35)$$

仅剩 1 组中继能源组件闲置；资源缺口向量为

$$(2, 0, 1, 2, 0, 0, 2, 0), \quad (8.36)$$

总缺口为 7 件。

表 8-3 问题四重复配置、库存闲置与库存缺口的分离核算

方案	duplicated 向量	合计	unused 向量	合计	deficit 向量	合计	CV
2 组	(0,1,0,0,1,0,1,1)	4	(0,0,0,0,0,0,0,2)	2	(0,1,0,0,1,0,1,0)	3	0.9243
3 组	(2,0,1,2,0,0,2,2)	9	(0,0,0,0,0,0,0,1)	1	(2,0,1,2,0,0,2,0)	7	1.0335

从上述结果可以看出，随着任务组数量由 2 增加至 3，重复配置总量由 4 件增加至 9 件，资源缺口则由 3 件增加至 7 件。说明在当前固定任务结构下，分区数量增加会使资源共享效率进一步下降。

这一现象可以从资源峰值的非可加性加以解释。统一资源池所需的峰值资源量通常满足

$$r_k^{\text{pool}} \leq \sum_h r_{hk}, \quad (8.37)$$

且当不同任务组的峰值时段并不完全重合时，上述不等式通常为严格不等式。统一资源池能够利用不同任务在时间轴上的错峰关系复用资源，而完全独立的任务组则失去这一优势。因此，任务组织越分散，需要为各组单独保留的峰值能力通常越多。

8.2.4 资源效率与工作量均衡的权衡

除资源配置外，问题四还需考虑不同任务组之间的工作量均衡性。根据固定运输任务工作量计算，两组方案的工作量变异系数为

$$CV_2 = 0.9243, \quad (8.38)$$

三组方案为

$$CV_3 = 1.0335. \quad (8.39)$$

两者均表现出较明显的任务负荷不均衡。两组方案中，S014 单独任务组仅承担 3 箱货物和 0.52 h 固定任务工作量，而另一组承担 77 箱货物和 13.19 h 工作量。三组方案虽然进一步增加了任务组数量，但仍存在一个承担 69 箱货物和 11.25 h 工作量的主任务组。

因此，“库存缺口最小”并不意味着“工作量最均衡”。这一结果与上一节采用的词典序目标一致：本文首先最小化资源缺口，其次减少总配置数量，最后才在前两项目标相同的方案之间比较工作量均衡性。换言之，当前选出的方案是资源约束优先意义下的分区，而不是单独追求任务量平均的方案。

表 8-4 问题四最小缺口分区的综合比较

方案	穷举划分数	库存缺口/件	配置总数/件	工作量变异系数	闲置组件/组
2 组	127	3	31	0.9243	2
3 组	966	7	36	1.0335	1

表 8-4 进一步汇总了两类方案。两组情形共完整枚举 127 种划分，最终方案资源缺口为 3 件，总配置量为 31 件，工作量变异系数为 0.9243；三组情形共枚举 966 种划分，最终方案资源缺口增加至 7 件，总配置量为 36 件，工作量变异系数为 1.0335。

从当前固定任务看，增加任务组数量并未改善最终入选方案的资源利用效率，反而同时增加了总配置量和资源缺口。其根本原因是问题三原方案高度依赖运输无人机、共享电池及中继资源在时间轴上的统一调度和错峰复用，而完全独立分区会切断这一共享机制。

需要指出，两组方案虽然在资源缺口、总配置规模和工作量变异系数三个指标上均优于当前入选的三组方案，但这并不意味着三组组织方式在任何管理目标下都没有意义。如果实际救援更加重视责任区域独立、指挥链简化或其他本文未纳入目标函数的组织因素，仍可从完整枚举结果中选择其他任务分区。此时，应同步报告该分区对应的新增资源需求，避免以组织独立性为代价却忽略装备配置约束。

综合而言，在问题三任务完全固定的条件下，统一资源池能够依靠任务之间的时间错峰实现较高的资源复用效率；一旦要求各任务组完全独立，这种共享优势将转化为新的重复配置和库存缺口。两组方案至少需要补充 3 件资源，三组方案至少需要补充 7 件资源，且最小缺口方案均存在较明显的工作量不均衡。因此，实际救援组织在选择分区粒度时，需要在任务独立性、资源配置成本和工作量均衡性之间进行综合权衡。

资源需求、分组工作量和分类型库存缺口的对比见图 8-5—8-7，用于直观呈现两组与三组方案的资源代价差异。

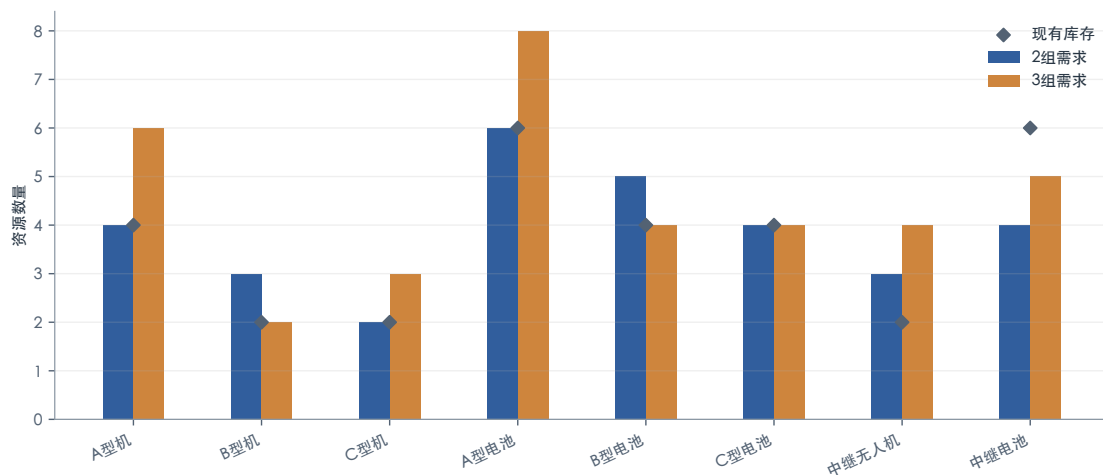


图 8-5 两组与三组方案的资源需求及现有库存比较

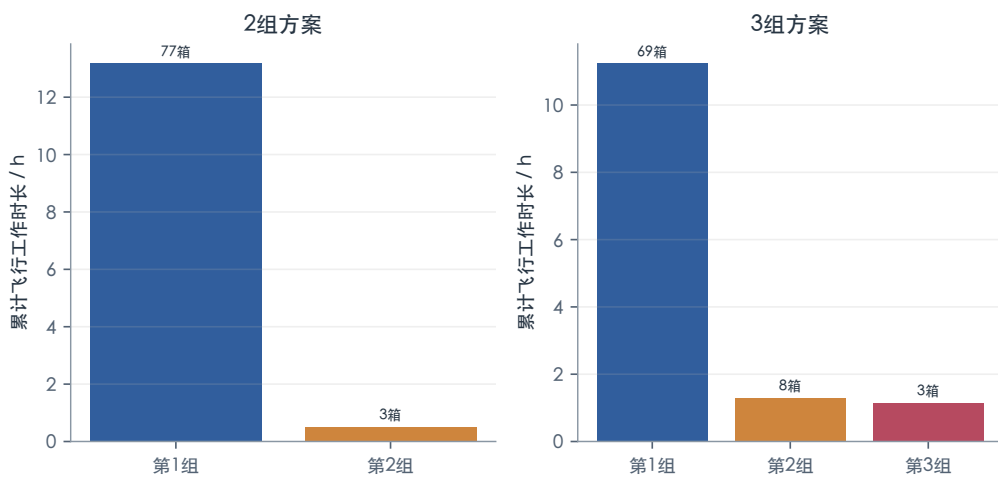


图 8-6 两组与三组方案的组间固定任务工作量

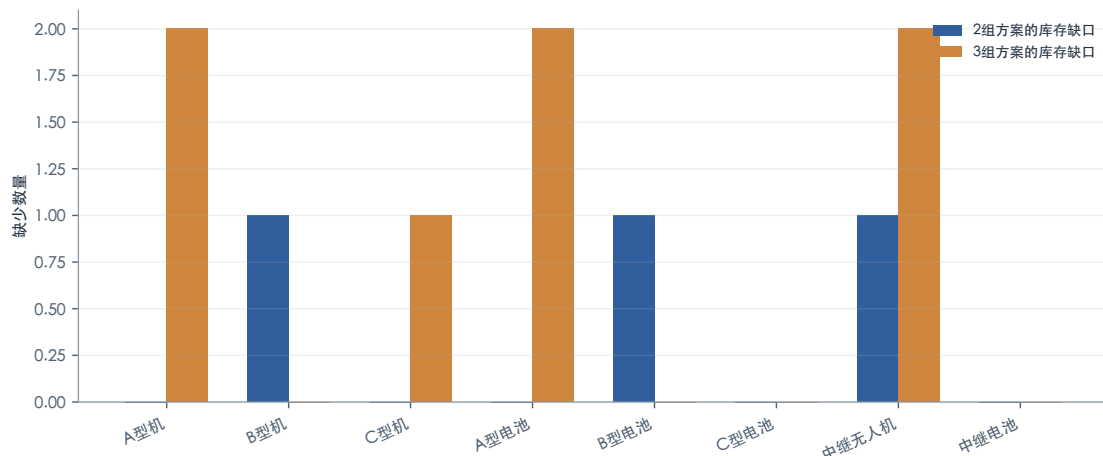


图 8-7 两组与三组方案的分类资源库存缺口

9. 敏感性与可行性检验

前述四个问题均基于题给设备参数、地形数据与通信参数进行建模和求解。然而，在实际山区洪涝救援中，返航安全余量、无人机实际能耗以及无线链路传播条件均可能相对于标称参数发生变化。因此，仅验证标称条件下的方案可行性还不足以判断方案的风险位置，还需要进一步识别哪些任务最接近约束边界，以及参数扰动达到何种程度时固定方案会首先失效。

本节不重新优化运输路线、中继位置和任务时刻，而是在保持各问正式方案不变的条件下开展固定方案压力检验。其目的不是证明方案在所有扰动下均具有鲁棒性，而是利用已有结果识别安全余量较小的关键任务，并明确当前方案从“标称可行”向“约束失效”转变的临界位置。

9.1 返航安全余量对单点组批的影响

问题一的单点直送模型要求任一运输架次在完成任务并返回调度中心后，仍保留规定比例的可用电量。记机型 g 的可用电量为 E_g^{use} ，返航安全余量为 ρ_g ，则任一由该机型执行的架次 k 必须满足

$$E_k \leq (1 - \rho_g) E_g^{\text{use}}. \quad (9.1)$$

当返航安全余量 ρ_g 提高时，单架次允许使用的最大能量 $(1 - \rho_g) E_g^{\text{use}}$ 随之下降。因此，在其他条件不变时，原有可行装载模式只能保持不变或退出可行集，不会由于安全余量提高而产生新的可行组批。这意味着最少架次数关于安全余量总体呈非减特征，但由于货箱不可拆且批次结构为离散决策，其变化通常表现为分段稳定和临界点跳变，而非连续变化。

对问题一正式模型进行返航安全余量扫描，结果如图 9-1 所示。当安全余量分别取 10% 和 20% 时，最少架次数均为 18 次，总能耗均为 59.1313 kWh。这表明在上述区间内，虽然单架次可用能量发生变化，但当前词典序最优方案中的关键批次仍未跨越能量可行边界，因此最优组批结构保持不变。

当安全余量提高至 30% 时，部分原有批次不再满足返航储备要求，最少架次数由 18 次增加至 20 次，总能耗相应上升至 67.2276 kWh。这一结果反映出不可拆货箱组批问题具有明显的阈值效应：在关键批次退出可行集之前，最优整数方案可以保持稳定；一旦某些高负载批次失去可行性，就必须重新拆分为更多架次，从而造成架次数和总能耗的离散跃迁。

当安全余量进一步提高至 40% 时，S004 的 14 kg 整箱饮用水在单点直接往返条件下已不存在能够满足能量约束的可行机型。由于该货箱不可拆分，此时单纯增加运输架次数不能恢复可行性。因此，40% 安全余量下出现的并不是一般意义上的“架次不足”，而是单个不可拆货箱已经超出当前机队在该服务区上的安全运输能力。

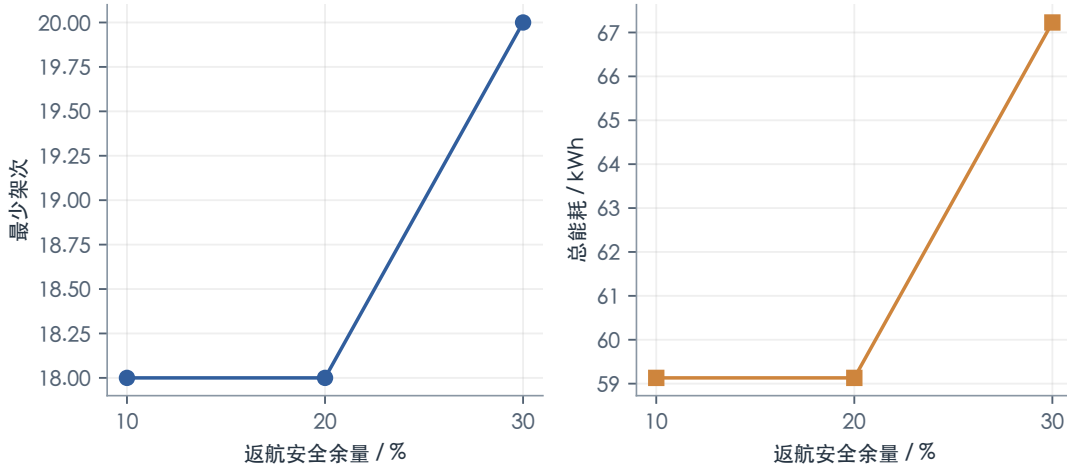


图 9-1 返航安全余量对单点组批架次和能耗的影响

由上述结果可以看出，返航储备比例不仅影响无人机的能量裕度，还会通过改变可行批次集合进一步影响整数组批结构。因此，实际任务中若提高安全余量，不能简单按照原方案同比缩减载荷，而应重新核验各服务区不可拆货箱的单架次可行性。

9.2 运输能耗扰动与关键航次识别

问题二和问题三的运输任务均采用“水平等效航程能耗 + 爬升势能”的统一能耗模型。在真实救援环境中，风场变化、设备状态以及标称参数误差都可能导致实际水平巡航能耗高于计算值。为识别固定运输方案中最接近能源安全边界的航次，本文保持已经选定的路线、机型、起飞时刻和资源配置不变，仅对所有水平飞行能耗施加统一倍率扰动。

记架次 k 的标称水平飞行能耗和爬升能耗分别为 E_k^{hor} 和 E_k^{climb} 。当水平能耗增加比例为 δ 时，扰动后的总能耗写为

$$E_k(\delta) = (1 + \delta)E_k^{\text{hor}} + E_k^{\text{climb}}. \quad (9.2)$$

若机型 g_k 的返航储备比例为 ρ_{g_k} ，则架次 k 触及安全储备边界时满足

$$(1 + \delta_k^{\text{crit}})E_k^{\text{hor}} + E_k^{\text{climb}} = (1 - \rho_{g_k})E_{g_k}^{\text{use}}. \quad (9.3)$$

因此，对水平能耗而言，该架次能够承受的临界相对增幅为

$$\delta_k^{\text{crit}} = \frac{(1 - \rho_{g_k})E_{g_k}^{\text{use}} - E_k^{\text{climb}} - E_k^{\text{hor}}}{E_k^{\text{hor}}}. \quad (9.4)$$

对固定方案中的全部航次分别计算 δ_k^{crit} ，其中最小值对应整个方案最敏感的运输任务。

计算结果表明，问题二和问题三的运输部分均由 A 型运输任务 T010 控制。该架次在标称状态下的电量裕度仅为 0.01829 kWh，水平巡航能耗只需增加 0.5441% 便会触及 20% 返航储备边界。若将扰动折算为总能耗增幅，对应临界值为 0.5107%。

问题一的最紧任务为 Q1-07，其水平能耗临界增幅为 4.1343%，对应总能耗临界增幅为 4.0155%。与之相比，问题二和问题三中的最紧架次明显更加接近能源约束边界，表明在加入多点访问和时序资源协调以后，部分高负载运输任务的安全余量被进一步压缩。

表 9-1 固定航次的能耗扰动临界值

方案	最紧架次	水平项临界增幅	同比总能耗临界增幅
问题一	Q1-07	4.1343%	4.0155%
问题二	T010	0.5441%	0.5107%
问题三	T010	0.5441%	0.5107%

为进一步观察固定方案在不同扰动幅度下的失效过程，将所有运输任务的水平能耗分别统一放大 1%、2%、3% 和 5%，并重新核验逐架次返航 SOC。结果见表 9-2。

表 9-2 固定运输方案的水平能耗倍率压力检验

方案	倍率	最低 SOC/%	越界架次	越界架次编号
问题二	1.00	20.41	0	—
问题二	1.01	19.66	1	T010
问题二	1.02	18.91	3	T010,T011,T017
问题二	1.03	18.17	3	T010,T011,T017
问题二	1.05	16.67	4	T010,T011,T017,T018
问题三	1.00	20.41	0	—
问题三	1.01	19.66	1	T010
问题三	1.02	18.91	3	T010,T011,T017
问题三	1.03	18.17	3	T010,T011,T017
问题三	1.05	16.67	4	T010,T011,T017,T018

在标称条件下，问题二和问题三运输方案的最低返航 SOC 均为 20.41%，所有运输任务满足 20% 返航储备要求。当水平能耗统一提高 1% 后，最低返航 SOC 下降至 19.66%，T010 成为首个越界航次。

当扰动提高至 2% 时，T010、T011 和 T017 三个航次同时越界，最低返航 SOC 下降至 18.91%；提高至 3% 后，越界航次集合仍为 T010、T011 和 T017，但最低 SOC 进一步下降至 18.17%。当扰动达到 5% 时，T018 也进入越界集合，此时共有 4 个运输架次不再满足 20% 返航储备要求，最低返航 SOC 降至 16.67%。

这一结果说明，固定运输方案的能源风险并非平均分布在所有航次上，而是集中于少数高负载、高爬升或长航程任务。因此，在实际执行中，对 T010 等临界航次进行起飞前电池状态和环境条件复核比仅关注全方案平均能耗更有意义。

需要强调的是，上述分析属于固定方案的压力检验，并没有在能耗参数发生变化后重新执行路线搜索和资源调度。某一航次越过 20% 返航储备线，只能说明当前路线、机型和时刻组合在该扰动水平下失去可行性，不能据此推出重新组批、换机型或重新调度以后整个问题仍无可行解。

9.3 通信链路附加损耗敏感性

问题三要求运输无人机在飞行与交接全过程中保持双向通信。正文采用连续区间证书证明每一段时间区间内直连链路或同一中继对应的接入链路和回程链路均满足损耗阈值。

为了评价当前固定通信配置对传播条件恶化的敏感程度，本节保持问题三中的中继位置、悬停高度、服务时窗和通信提供方不变，在原连续通信证书的链路损耗上统一叠加额外损耗 ΔL 。

设连续区间 I 的原保守损耗上界为 $\bar{L}_I$ ，对应链路允许的最大损耗为 $L_I^{\max}$ ，则该区间的标称证明裕度为

$$\mu_I = L_I^{\max} - \bar{L}_I. \quad (9.5)$$

加入额外链路损耗 ΔL 后，剩余证明裕度变为

$$\mu'_I = \mu_I - \Delta L. \quad (9.6)$$

当

$$\mu'_I \geq 0 \quad (9.7)$$

时，原连续区间证书在该附加损耗水平下仍然成立；若

$$\mu'_I < 0, \quad (9.8)$$

则只能说明当前链路分配对应的原证书不再足以证明该区间可行。

问题三主方案共包含 4061 个连续通信区间。在标称参数下，全部区间均获得可行性证书，最小证明裕度为 0.093780 dB。这一数值说明最紧链路距离传播损耗阈值已经很近。

表 9-3 固定链路分配下额外损耗对连续证书的影响

额外链路损耗/dB	未获证区间数	最低剩余裕量/dB
0.0	0	0.094
0.5	36	-0.406
1.0	170	-0.906
2.0	671	-1.906
3.0	1101	-2.906

表 9-3 显示，当额外链路损耗为 0 时，全部 4061 个区间均通过证明；当附加损耗达到 0.5 dB 时，已有 36 个区间无法继续由原证书证明，最低剩余裕度下降为 -0.406 dB。当附加损耗分别提高至 1.0、2.0 和 3.0 dB 时，未获证区间数量进一步增加至 170、671 和 1101 个。

由于标称条件下的最小证明裕度仅为 0.093780 dB，因此额外链路损耗增加约 0.1 dB 时，现有连续通信证书至少会有一个区间失效。这一结果说明当前主方案虽然在题给标称传播条件下满足连续通信要求，但最紧链路的剩余裕度较小。

需要区分“未获证”与“通信问题无可行解”。本节没有重新选择中继位置、调整中继服务时间或重新协调运输任务，所以某些原证书在附加损耗后失效，只说明既定通信配置已不能继续通过原证明方式认证。若允许重新布置中继、改变服务窗口或重新调度运输任务，仍可能得到新的通信可行方案。

9.4 主方案的全流程可行性回放

除参数扰动外，还需要确认正式主方案在标称条件下能够同时满足货箱交付、时间窗、能源和共享资源等全部约束。因此，本文对问题二和问题三的方案进行了逐箱、逐航次、逐资源和逐通信区间的独立回放。

在货箱交付方面，逐一按照每个货箱的实际交接完成时刻重新计算其时限约束。问题二最紧硬截止仍提前 221.03 s，问题三最紧硬截止提前 84.65 s。两份正式方案中，31 箱适用硬截止的货物全部在截止时间前完成交接，64 箱非医疗物资不存在迟到，同时未发现错站或重复交付。

在能源方面，问题二和问题三运输任务的最低返航 SOC 均为 20.4065%，高于 20% 返航储备线。问题三中继无人机的最低返航 SOC 为 38.6085%，亦满足能源约束。

在资源时序方面，对 8 架运输无人机、各型号共享电池、2 架中继无人机和 6 组中继能源组件的占用区间重新扫描，机体、电池及中继资源均不存在不允许的时间重叠。因此，正式方案不仅在任务层面满足配送要求，也能够在题给实体资源数量下按当前时间表执行。

在通信方面，问题三的 4061 个连续通信区间全部具有对应链路证书，最小证明裕度为 0.093780 dB。由此，问题三主方案在标称参数下完成了“货箱-时限-能源-资源-通信”五个层面的闭环可行性核验。

9.5 通信裕度候选与能源指标边界

在问题三的有限协调审计中，除最终主方案外还保留了其他非支配候选。其中最大化最小通信裕度的候选方案将最小链路裕度由 0.093780 dB 提高至 0.094145 dB，绝对增加量仅为

$$0.094145 - 0.093780 = 0.000365 \text{ dB}. \quad (9.9)$$

与此同时，该候选的总能耗增加至 79.0401 kWh。由于 0.000365 dB 的改善远小于前述 0.1 dB 附加损耗压力尺度，因此不能将其解释为通信稳健性已经得到实质提升。这一比较的意义主要在于说明：在有限候选排序中可以显式加入最小链路裕度指标，但当前候选集合尚未产生具有明显裕度优势的新方案。

问题三主方案总能耗为 78.8732 kWh。

按全部 758 kg 货物计算，单位载货能耗为 0.10405 kWh/kg。有限候选中的节能对照方案可减少 0.0776 kWh 能耗，但联合任务收尾时间延后 55.88 s，说明在当前候选集合内，任务完成时间与能耗之间存在一定折中关系。

需要注意，上述能耗指标仅代表无人机任务执行阶段的能源消耗。题目附件未提供电池充电效率、电网排放因子以及设备制造、维护和退役阶段的全生命周期数据，因此不能依据本文的 kWh 结果直接计算碳排放量或碳减排量。

此外，本文采用静态 DEM 和题给标称设备参数，没有进一步模拟强风、持续降雨、电池衰减、定位误差及灾后新增滑坡等随机因素。因此，当前敏感性分析主要用于识别固定方案中的薄弱环节，而不是证明当前方案在任意现场条件下均保持可行。

为便于汇总，表 9-4 列出了本文已经完成的主要参数扰动及对应结论。

表 9-4 已完成的参数扰动及固定方案敏感性结论

扰动对象	检验范围或阈值	已验证影响
Q1 返航安全余量	10%–40%	30% 增至 20 架次；40% 出现不可直送整箱
Q2 水平能耗	统一增加 0.5441%	最紧架次触及 20% 返航储备
Q3 水平能耗	统一增加 0.5441%	最紧架次触及 20% 返航储备
Q3 额外链路损耗	增加 0.1 dB	现有连续通信证书至少一段失效

由表 9-4 可知，问题一主要风险表现为返航储备提高后可行批次集合缩小；问题二和问题三的运输部分主要受少数临界航次的能源裕度限制；问题三通信部分则受最紧连续链路较小的损耗裕度限制。

因此，在实际部署前，应优先依据最新天气条件、设备状态和实测通信质量重新核验这些临界任务。若现场参数已经超过本文识别出的固定方案安全边界，则应重新执行运输和中继调度，而不应机械沿用标称条件下的原时间表。

10. 模型评价与结论

本文围绕山区洪涝灾害条件下的无人机应急物资投送与通信保障问题，依次研究单点直送能力、异构多航次运输、连续通信协同以及固定联合任务的独立分区。四个问题并非彼此割裂，而是在同一物理计算体系下逐层增加现实约束，形成

$$\text{单点组批} \rightarrow \text{多航次运输} \rightarrow \text{连续通信} \rightarrow \text{独立分区} \quad (10.1)$$

的完整决策链。

全文始终采用统一的节点坐标、DEM 地形数据、航段时间计算、载荷相关能耗、返航安全储备和能源周转规则。因此，后一问对前一问结果的继承不会混入模型口径变化，四个问题的结果可以在同一物理尺度下进行比较。

10.1 模型的主要特点

10.1.1 统一物理模型贯穿四个问题

本文没有针对四问分别采用互不一致的飞行时间和能耗近似，而是从原始 DEM 和节点数据出发，统一完成空间距离、地形净空高度、逐航段载荷、飞行时间和能量消耗计算。

尤其在多点运输中，无人机完成货箱交接后剩余载荷会发生变化，因此后续航段能耗并非由一个固定平均载荷决定。本文按照实际服务顺序更新每一段飞行载荷，使路线结构与能耗之间保持直接对应关系。

统一物理口径的另一个作用是保证跨问题结果可比。例如，问题二加入实体无人机和共享电池后，问题三继续沿用完全相同的运输能耗与资源周转规则，仅进一步增加通信约束；问题四又完全固定问题三任务，仅分析资源池被拆分后的配置变化。这种递进关系使每一问新增的代价均能对应到新加入的约束。

10.1.2 根据问题结构选择不同求解层次

四个问题的决策空间具有明显不同的结构，因此本文没有强行采用同一种算法处理全部问题。

问题一中，每个服务区的货箱种类与数量有限，单点直送批次可以完整枚举。在此基础上，采用计数状态动态规划遍历全部可行整数组批状态，从而在限定单点模型内得到词典序精确解。

问题二加入多服务区、多航次、实体无人机、共享电池和硬时限以后，路线结构与时间资源调度高度耦合。若直接把所有货箱分配、访问顺序和资源时序同时放入一个完整模型，组合规模会显著扩大。因此，正文将其分为两个层次：上层通过路线列启发式搜索货箱组合、访问顺序和机型，下层在路线固定后利用 CP-SAT 确定实体无人机、电池和任务开始时刻。

这一分解使路线构造与资源调度职责清晰：启发式部分负责探索离散路线结构，CP-SAT 则负责检验和安排固定路线下的时序资源可行性。同时，本文明确区分概念上的完整 VRPTW 模型与实际计算中的两层实现，避免将有限路线搜索得到的方案误写为全局最优。

问题三的新增困难来自全过程连续通信。若仅在若干离散时刻检查链路，无法严格排除两个采样点之间出现瞬时地形遮挡或损耗越界。因此，本文使用区间距离上界和地形遮挡上界构造连续通信证书，将“若干时刻可通信”提升为“整个时间区间均得到证明”。

在运输与通信协同方面，当前程序对有限运输族和中继窗口进行分解协调审计，没有把通信不可行信息以冲突割形式迭代返回运输主问题。因此，本文只报告有限候选空间内的协调结果，不扩大其理论结论。

问题四则具有另一种结构。当问题三的路线、时刻和通信关系全部固定以后，服务区之间的不可拆关系可以转化为图的连通分量，最终得到 8 个不可拆任务组件。两组和三组

划分分别只有 127 种和 966 种，因此可以进行完整枚举，并精确计算固定任务时间轴上的资源峰值。

10.1.3 从目标值报告扩展到可执行性证书

本文不仅比较架次数、完成时间和能耗，还进一步对正式方案进行逐项回放。

在问题二和问题三中，每个货箱均根据实际交接时刻重新核验目的地和时间约束；每个运输架次独立复算能耗和返航 SOC；实体无人机和共享电池按照完整占用区间检查时间重叠；问题三的中继无人机和能源组件采用相同方式复核；通信部分则对全部 4061 个连续区间保存证明结果。

因此，本文最终给出的主方案不是只有目标函数数值的抽象解，而是能够从“货箱-路线-时刻-实体资源-能源-通信”逐层回放的可执行方案。

10.2 四个问题的主要结论

10.2.1 问题一：单点直送与不可拆货箱组批

问题一首先确定不同服务区和机型组合下的最大安全载荷，并在此基础上枚举不可拆货箱的可行直送批次。

在 20% 返航安全余量下，完成 15 个服务区共 80 箱物资配送需要 18 个运输架次，总能耗为 59.1313 kWh。在本文设定的单点直接往返、固定航迹和载荷能耗模型范围内，计数状态动态规划覆盖全部可行整数模式，因此该结果具有相应词典序目标下的精确最优性。

安全余量分析进一步表明，10% 和 20% 储备下最优组批保持不变；提高至 30% 后，最少架次数增加至 20 次，总能耗升至 67.2276 kWh；当储备提高至 40% 时，S004 的 14 kg 整箱饮用水已不存在单点直送可行机型。

因此，问题一不仅给出了标称安全余量下的组批方案，还识别出单点直送能力随安全储备收紧时的关键可行边界。

10.2.2 问题二：异构多航次运输与共享资源调度

问题二在问题一基础上允许单架次访问多个服务区，同时加入物资硬时限、实体运输无人机、共享电池以及充电周转。

实际求解采用“路线列启发式 + 固定路线 CP-SAT”两层框架。不同路线搜索方法在统一初始条件和固定评估预算下进行比较，所有候选均通过同一物理规则与 CP-SAT 排程接口重新核验。

最终保留的 24 架次可执行方案完成全部 80 箱配送，全部运输无人机于 6276.83 s，即 104.61 min 内返航，总能耗为 71.9169 kWh。

31 箱具有硬截止要求的物资全部按时交付，64 箱非医疗物资的加权迟到为 0；最终一箱货物于 5734.23 s 完成交接。最坚硬截止仍提前 221.03 s，最低运输机返航 SOC 为 20.4065%。同时，8 架实体运输无人机和 14 组各型电池在完整调度时间轴上均不存在非法资源重叠。

需要说明，该 24 架次方案来源于共享历史可行方案并经过统一协议和完整回放保留。由于上层路线集合采用有限启发式搜索，本文将其表述为当前计算条件下的高质量可执行方案，而不将其作为完整异构 VRPTW 的全局最优证明。

10.2.3 问题三：运输-通信协同与连续链路认证

问题三进一步要求运输无人机在起飞、飞行、服务区交接和返航全过程中保持双向通信。

针对地形遮挡下离散时间采样可能遗漏短时失效的问题，本文通过距离上界和遮挡上界为每一个通信区间构造连续证书。若采用中继，则同一中继必须在整个对应区间内同时满足运输机至中继的接入链路以及中继至固定网关的回程链路约束。

在当前三类运输族和 32 个中继窗口候选中完成有限协调审计后，最终主方案由 26 个运输架次和 3 个中继架次组成。联合任务于 6769.22 s，即 112.82 min 完成，总能耗为 78.8732 kWh。

最后一箱货物于 6317.40 s 完成交付；31 箱硬截止均满足，其中最紧截止仍提前 84.65 s。运输任务最低返航 SOC 为 20.4065%，中继任务最低返航 SOC 为 38.6085%。

全部 4061 个连续通信区间均获得证书，最小链路证明裕度为 0.093780 dB。有限候选中最大化最小通信裕度的方案只能将其提高至 0.094145 dB，增幅为 0.000365 dB，同时总能耗升至 79.0401 kWh。因此，当前有限搜索并未得到具有实质通信裕度提升的替代方案。

与问题二相比，加入连续通信要求以后，运输架次数由 24 增加至 26，并额外增加 3 个中继架次；联合收尾时间由 104.61 min 增加至 112.82 min，总能耗由 71.9169 kWh 增加至 78.8732 kWh。这些变化量化了在当前有限候选集合中为保证连续通信所付出的运输与能源代价。

10.2.4 问题四：固定联合任务的独立分区

问题四保持问题三主方案中的箱组、服务区顺序、运输时刻和通信关系全部不变，根据多点运输路线构造服务区关联图，将 15 个服务区归并为 8 个不可拆任务组件。

对 8 个组件的两组和三组划分分别进行完整枚举。在“库存缺口-总配置规模-工作量均衡”的词典序目标下，两组方案将 S014 单独划为一组，其余 14 个服务区组成另一组；三组方案则将 S003 和 S012 分别单独成组，其余 13 个服务区组成主任务组。

两组完全独立运行时，至少还需增加 1 架 B 型运输无人机、1 组 B 型电池和 1 架中继无人机，总资源缺口为 3 件。

三组独立运行时，至少还需增加 2 架 A 型运输无人机、1 架 C 型运输无人机、2 组 A 型电池和 2 架中继无人机，总资源缺口增加至 7 件。

两组和三组方案的重复配置总量分别为 4 件和 9 件，工作量变异系数分别为 0.9243 和 1.0335。结果说明，随着独立任务组数量增加，统一资源池原有的跨任务错峰复用能力进一步下降，从而产生更多重复配置与库存缺口。

同时，最小资源缺口方案并不等价于工作量最均衡方案。这是本文词典序目标优先保证资源可实施性的直接结果，反映了任务独立性、设备配置成本与组织均衡之间的现实权衡。

10.3 四问结果汇总与结论边界

表 24 汇总四个问题的核心结果以及对应的结论适用范围。

表 10-1 四问主要结果及结论适用范围

问题	主要结果	结论适用范围
一	18 架次, 59.1313 kWh	单点直送、固定航迹及本文能耗模型内的词典序精确解
二	24 架次, 104.61 min, 71.9169 kWh	有限路线搜索所得、经完整回放验证的可执行方案，不构成完整 VRPTW 全局最优证明
三	26 个运输架次 +3 个中继架次, 112.82 min, 78.8732 kWh	三类运输族和 32 个中继窗口候选内的有限协调结果，4061 个连续通信区间全部获证
四	两组缺 3 件资源；三组缺 7 件资源	固定问题三联合任务后的独立分区资源峰值，不包含运输路线与任务时刻重排

从表 10-1 可以看到，不同问题具有不同的最优性或可行性结论边界。问题一由于有限

状态空间被完整覆盖,可以给出限定模型内的精确最优结论;问题二和问题三涉及规模更大的组合搜索,其正式方案应表述为有限搜索和统一回放下的可执行结果;问题四在固定问题三任务以后重新形成有限组合空间,因此可以完整枚举两组和三组的全部分区。

明确区分上述结论层级,可以避免把“固定子问题精确求解”错误扩大为“完整原问题全局最优”,也避免将标称参数下的可行性直接解释为真实灾害环境中的稳健保证。

10.4 模型的局限性

尽管本文在同一框架下处理了运输、能源、共享资源和连续通信等多类约束,仍存在若干需要明确的适用边界。

第一,问题二的路线层采用有限启发式搜索。固定路线以后,CP-SAT 能够严格处理实体无人机、电池和任务时刻之间的资源冲突,但其无法弥补上层路线集合没有覆盖的组合。因此,当前 24 架次方案不构成完整异构 VRPTW 的全局最优证明。

第二,问题三的运输-通信协调仍属于有限候选协调。当前程序能够对运输候选执行连续通信核验,并在有限的中继位置与服务窗口中搜索修复方案,但尚未实现通信不可行信息以冲突割形式返回运输主问题的迭代机制。因此,通信约束尚未完全参与运输路线生成过程。

第三,本文通信模型依据题给 2.4 GHz 自由空间传播模型、DEM 遮挡和固定附加损耗建立。现实山区可能同时受到降雨、植被、建筑、多径、天线姿态及电磁干扰等因素影响,这些因素没有在当前题目数据下进一步建模。

第四,运输模型采用静态 DEM 和确定性飞行参数。强风、设备老化、定位误差和灾后新增滑坡均可能改变实际飞行时间、能源消耗和有效地形净空。第 9 章的压力检验已经表明,最紧运输航次的能量裕度较小,因此实际任务前仍需要重新认证。

第五,问题四严格固定问题三正式方案以后再执行独立分区。这与题目要求一致,但相应的 3 件和 7 件资源缺口只代表“固定联合任务被拆分”后的独立配置代价。如果允许分区以后重新设计路线、机型和中继任务,资源需求可能随之变化,该情形不属于当前问题四的求解范围。

10.5 工程启示与最终结论

综合四问结果可以看到,山区洪涝灾害中的无人机救援并不是单一的最短路径或最少架次问题。地形净空、货箱不可拆、载荷相关能耗、返航储备、硬时限、共享电池以及通信覆盖之间存在明显耦合。

问题一说明,即使不考虑共享机体和电池时序,地形和返航储备已经能够显著限制不可拆货箱的单点运输能力。

问题二进一步表明,允许多服务区合并运输以后,仍必须同时处理实体无人机和电池的时序复用,否则仅在路线层面可行的方案未必能够实际执行。

问题三表明,物流可行并不等价于通信可行。在受地形遮挡的山区环境中,保证全过程通信需要额外的中继任务和运输协调,从而增加联合任务完成时间与能源消耗。

问题四则揭示了另一类资源代价:即使统一调度下现有库存能够完成问题三主方案,一旦进一步要求各任务组完全独立,原有的错峰共享优势就会消失,并产生新的重复配置和资源缺口。

敏感性分析进一步指出,当前方案的主要风险集中在少数临界任务。问题二和问题三最紧运输航次的水平能耗仅允许约 0.5441% 的增幅。

问题三最小连续通信裕度仅为 0.093780 dB。因此,当前结果应被定位为题给标称参数下经过逐箱、逐航次、逐资源和逐通信区间核验的“可计算、可执行、可回放”决策基准,而不应直接解释为复杂真实灾害环境中的无条件稳健最优方案。

实际救援执行前,应结合实时天气、无人机与电池健康状态、最新地形信息和实测通信链路质量,重点重新核验本文识别出的临界航次和最紧通信区间。若现场参数已经超出

固定方案的安全边界，则应重新执行运输与中继调度，而不是机械沿用标称条件下的既定时间表。

参考文献

- [1] 新华社. 记者手记: 抵近广西横州镇龙乡 [EB/OL]. 2026-07-09. <https://www.xinhuanet.com/politics/20260709/acf8e4b353304bb78007d8224f1cd2ef/c.html>.
- [2] 新华每日电讯. 三进“孤岛乡”[EB/OL]. 2026-07-12. <https://www.news.cn/local/20260712/8bd1f64af3124569b5cf4415fc013acd/c.html>.
- [3] Copernicus Data Space Ecosystem. Copernicus DEM—Global and European Digital Elevation Model[DB/OL]. DOI:10.5270/ESA-c5d3d65. <https://dataspace.copernicus.eu/explore-data/data-collections/copernicus-contributing-missions/collections-description/COP-DEM>, accessed 2026-09-26.
- [4] DJI Enterprise. Matrice 350 RTK Specifications[EB/OL]. <https://enterprise.dji.com/matrice-350-rtk/specs>, accessed 2026-09-26.
- [5] Doodle Labs. Doodle Labs releases Sense, new interference-avoidance features for Mesh Rider Radio[EB/OL]. 2023-10-16. <https://doodlelabs.com/news/sense-interference-avoidance-release/>.
- [6] Dorling K, Heinrichs J, Messier G G, Magierowski S. Vehicle Routing Problems for Drone Delivery[J]. IEEE Transactions on Systems, Man, and Cybernetics: Systems, 2017, 47(1):70–85. DOI:10.1109/TSMC.2016.2582745.
- [7] Zhang J, Campbell J F, Sweeney D C II, Hupman A C. Energy consumption models for delivery drones: A comparison and assessment[J]. Transportation Research Part D: Transport and Environment, 2021, 90:102668. DOI:10.1016/j.trd.2020.102668.
- [8] Texas Instruments. Li-Ion Battery Charger Solution Using an MSP430 MCU: Application Report SLAA287B[EB/OL]. 2005, revised 2022-04. <https://www.ti.com/lit/an/slaa287b/slaa287b.pdf>.
- [9] International Telecommunication Union. Recommendation ITU-R P.525-5: Calculation of Free-Space Attenuation[S/OL]. 2024. <https://www.itu.int/rec/R-REC-P.525-5-202411-I/en>.
- [10] Zeng Y, Zhang R, Lim T J. Wireless Communications with Unmanned Aerial Vehicles: Opportunities and Challenges[J]. IEEE Communications Magazine, 2016, 54(5):36–42. DOI:10.1109/MCOM.2016.7470933.
- [11] Dantzig G B, Ramser J H. The Truck Dispatching Problem[J]. Management Science, 1959, 6(1):80–91. DOI:10.1287/mnsc.6.1.80.
- [12] Solomon M M. Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints[J]. Operations Research, 1987, 35(2):254–265. DOI:10.1287/opre.35.2.254.
- [13] Toth P, Vigo D, eds. Vehicle Routing: Problems, Methods, and Applications[M]. 2nd ed. Philadelphia: SIAM, 2014. DOI:10.1137/1.9781611973594.

- [14] Macrina G, Di Puglia Pugliese L, Guerriero F, Laporte G. Drone-aided routing: A literature review[J]. *Transportation Research Part C: Emerging Technologies*, 2020, 120:102762. DOI:10.1016/j.trc.2020.102762.
- [15] Murray C C, Chu A G. The Flying Sidekick Traveling Salesman Problem: Optimization of Drone-Assisted Parcel Delivery[J]. *Transportation Research Part C: Emerging Technologies*, 2015, 54:86–109. DOI:10.1016/j.trc.2015.03.005.
- [16] Kirkpatrick S, Gelatt C D Jr, Vecchi M P. Optimization by Simulated Annealing[J]. *Science*, 1983, 220(4598):671–680. DOI:10.1126/science.220.4598.671.
- [17] Glover F. Future Paths for Integer Programming and Links to Artificial Intelligence[J]. *Computers & Operations Research*, 1986, 13(5):533–549. DOI:10.1016/0305-0548(86)90048-1.
- [18] Osman I H. Metastrategy Simulated Annealing and Tabu Search Algorithms for the Vehicle Routing Problem[J]. *Annals of Operations Research*, 1993, 41:421–451. DOI:10.1007/BF02023004.
- [19] Google. OR-Tools CP-SAT Solver Documentation[EB/OL]. https://developers.google.com/optimization/cp/cp_solver, accessed 2026-09-26.
- [20] Feo T A, Resende M G C. Greedy Randomized Adaptive Search Procedures[J]. *Journal of Global Optimization*, 1995, 6:109–133. DOI:10.1007/BF01096763.

附录 A 核心代码与复现说明

A.1 问题一：组批模式枚举与计数状态动态规划

以下为项目实际运行的 Python 源码节选，直接从对应模块载入并保留源文件行号。此段包含返航储备下的安全载荷搜索、可行组批模式枚举、带缓存的计数状态动态规划及路线恢复。

```
12 def q1(m, rho=None, objective='count'):  
13     if rho is None:  
14         reserves={t['rho'] for t in m.types.values()}  
15         if len(reserves)!=1: raise ValueError('Q1 default reserve requires equal model reserve  
percentages')  
16         rho=reserves.pop()/100  
17     allroutes=[]; capacities=[]; certs=[]  
18     for node in range(1,16):  
19         for g,t in m.types.items():  
20             lo,hi=0.,float(t['Q'])  
21             for _ in range(optimization_parameters()['q1_capacity_bisection_iterations']):  
22                 mid=(lo+hi)/2  
23                 if m.energy(g,0,node,mid)+m.energy(g,node,0,0)<=(1-rho)*t['E']: lo=mid  
24                 else: hi=mid  
25                 capacities.append(dict(node=node,g=g,rho=rho,maxload=lo))  
26             ids=[i for i,b in enumerate(m.bboxes) if b['node']==node]  
27             kinds=list(dict.fromkeys(m.bboxes[i]['kind'] for i in ids))  
28             buckets=[[i for i in ids if m.bboxes[i]['kind']==k] for k in kinds]  
29             counts=tuple(map(len,buckets)); patterns=[]  
30             for sizes in product(*(range(c+1) for c in counts)):  
31                 if not any(sizes): continue  
32                 chosen=[b for bucket,n in zip(buckets,sizes) for b in bucket[:n]]  
33                 for g in m.types:  
34                     r=m.route(g,chosen,[node],rho)  
35                     if r:  
36                         cost=(1,r['energy'],r['duration']) if objective=='count' else ((r['energy'],  
37 ],1,r['duration']) if objective=='energy' else (r['duration'],1,r['energy']))  
38                         patterns.append((sizes,g,cost))  
39                 @lru_cache(None)  
40                 def dp(state):  
41                     if not any(state): return (0.,0.,0.),()  
42                     best=(float('inf'),)*3; path=()  
43                     first=next(i for i,v in enumerate(state) if v)  
44                     for j,(sz,g,cost) in enumerate(patterns):  
45                         if sz[first]==0 or any(a>b for a,b in zip(sz,state)): continue  
46                         val,tail=dp(tuple(b-a for a,b in zip(sz,state)))  
47                         value=tuple(a+b for a,b in zip(val,cost))  
48                         if value<best: best=value; path=(j,)+tail  
49                     return best,path  
50                 value,path=dp(counts)  
51                 if not math.isfinite(value[0]):  
52                     return dict(routes=[], capacities=capacities, certificates=certs, summary=dict(feasible=  
53 False, failed_node=node, status=2, message='Exact count-state dynamic program infeasible'))  
54                 offsets=[0]*len(counts)  
55                 for j in path:  
56                     sizes,g,_=patterns[j]; chosen=[]  
57                     for k,n in enumerate(sizes): chosen.extend(buckets[k][offsets[k]:offsets[k]+n]);  
58                     offsets[k]+=n  
59                     allroutes.append(m.route(g,chosen,[node],rho))  
60                 certs.append(dict(node=node, status=0, gap=0., candidates=len(patterns), states=dp.cache_info()  
61 ().currsz, method='exact_count_state_DP'))  
62     return dict(routes=allroutes, capacities=capacities, certificates=certs, summary=dict(count=len(  
allroutes), energy=sum(r['energy'] for r in allroutes), time=sum(r['duration'] for r in  
allroutes)))
```

A.2 问题二：路线搜索与固定路线排程核心代码

以下节选展示项目实际使用的资源解码、多策略搜索和固定路线 CP-SAT 排程实现。完整函数依赖同一模块中的路线变换、指标计算和验证函数。

A.2.1 资源解码与实体资源分配

```
32 def decode(m, plan):
33     if sorted(b for _, bs, _ in plan for b in bs) != list(range(len(m.bboxes))): return None
34     units={g:[0.]*len(m.units[g]) for g in m.types}
35     batteries={g:[0.]*m.types[g]['batteries'] for g in m.types}
36     out=[]
37     for index, (g, boxes, order) in enumerate(plan):
38         required={m.bboxes[b]['node'] for b in boxes}
39         if not boxes or len(order)>optimization_parameters()['route_search']['max_stops_per_route'] or len(order)!=len(set(order)) or set(order)!=required: return None
40         r=m.route(g, list(boxes), list(order))
41         if r is None: return None
42         ui=min(range(len(units[g])), key=lambda k: units[g][k])
43         bi=min(range(len(batteries[g])), key=lambda k: batteries[g][k])
44         start=max(units[g][ui], batteries[g][bi])
45         r.update(id=f'T{index+1:03}', start=start, end=start+r['duration'], unit=m.units[g][ui],
46               battery=f'{g}B{bi+1:02}')
47         units[g][ui]=r['end']; batteries[g][bi]=r['end']+charge(r['soc'], m.types[g]['charge'])
48         out.append(r)
49     return out
```

A.2.2 四类方法共用的路线搜索框架

```
136 def search(m, method, initial_plan, seed, budget, max_evaluations=None,
137            profile='timeliness', epsilon=None):
138     rng=random.Random(100000+seed)
139     start=time.monotonic();limit=start+budget
140     curr_plan=list(initial_plan);curr_routes=decode(m,curr_plan)
141     if curr_routes is None:raise RuntimeError('invalid initial plan')
142     rank=lambda routes:profile_score(m,routes,profile,epsilon)
143     curr_score=rank(curr_routes);best_plan=list(curr_plan);best_score=curr_score
144     seen={signature(curr_plan)};tabu={};history=[];iterations=0;evaluated=1
145     while time.monotonic()<limit and (max_evaluations is None or evaluated<max_evaluations):
146         iterations+=1
147         if method=='grasp':
148             # 随机化构造重启属于独立的搜索策略类别。
149             fresh=construct(m,seed*10000+iterations,multi=True)
150             if fresh is None:continue
151             q=plan_from_routes(fresh);routes=decode(m,q)
152             if routes is None:continue
153             z=rank(routes);evaluated+=1;curr_plan,curr_score=q,z
154         elif method=='hill':
155             candidates=[]
156             remaining=(max_evaluations-evaluated) if max_evaluations is not None else optimization_parameters()['route_search']['hill_neighbors']
157             for _ in range(min(optimization_parameters()['route_search']['hill_neighbors'],remaining)):
158                 q,routes=mutate(m,curr_plan,rng)
159                 if q is not None:
160                     candidates.append((rank(routes),q));evaluated+=1
161                 if not candidates:continue
162                 z,q=min(candidates,key=lambda x:x[0])
163                 if z<curr_score:curr_plan,curr_score=q,z
164                 elif iterations%optimization_parameters()['route_search']['hill_restart_interval']==0:curr_plan=list(best_plan);curr_score=best_score
165         elif method=='anneal':
166             q,routes=mutate(m,curr_plan,rng)
167             if q is None:continue
168             z=rank(routes);evaluated+=1
169             progress=(evaluated/max_evaluations) if max_evaluations else ((time.monotonic()-start)/budget)
170             temp=max(optimization_parameters()['route_search']['anneal_min_temperature'],1-progress)
171             delta=lexicographic_delta(z,curr_score)
172             scale=optimization_parameters()['route_search']['anneal_lexicographic_scale']
173             if delta<=0 or rng.random()<math.exp(-min(700,delta/(scale*temp))):
174                 curr_plan,curr_score=q,z
175         elif method=='tabu':
176             candidates=[]
177             remaining=(max_evaluations-evaluated) if max_evaluations is not None else optimization_parameters()['route_search']['tabu_neighbors']
178             for _ in range(min(optimization_parameters()['route_search']['tabu_neighbors'],remaining)):
179                 q,routes=mutate(m,curr_plan,rng)
180                 if q is None:continue
181                 sig=signature(q);z=rank(routes);evaluated+=1
182                 if tabu.get(sig,0)<=iterations or z<best_score:candidates.append((z,q,sig))
183                 if not candidates:continue
184                 z,q,sig=min(candidates,key=lambda x:x[0]);curr_plan,curr_score=q,z
185                 tabu[sig]=iterations+optimization_parameters()['route_search']['tabu_tenure']
186             if curr_score<best_score:
187                 best_plan=list(curr_plan);best_score=curr_score
188                 history.append({'iteration':iterations,'elapsed_s':round(time.monotonic()-start,3),'score':best_score})
189             routes=decode(m,best_plan)
190     guard_hit=time.monotonic()>=limit and (max_evaluations is None or evaluated<max_evaluations)
191     return dict(method=method,seed=seed,profile=profile,budget_s=budget,
192                evaluation_limit=max_evaluations,wall_clock_guard_hit=guard_hit,
193                elapsed_s=time.monotonic()-start,iterations=iterations,
194                valid_candidates_evaluated=evaluated,
195                heuristic=dict(**metrics(m,routes),minimum_energy_margin=minimum_energy_margin(m,routes)),
196                history=history,plan=best_plan)
```

A.2.3 固定路线的分阶段 CP-SAT 排程

```
207 def refine_timefirst(m, rs, seconds=6.0):
208     """医疗及首批硬截止通过后，依次压低其他物资迟到、返航收尾与交付时刻。"""
209     from ortools.sat.python import cp_model
210     model=cp_model.CpModel();starts=[];ends=[];intervals=[];battery_intervals=[]
211     tardy_terms=[];arrival_terms=[];horizon=optimization_parameters()['route_search']['
        schedule_horizon_s']
212     for j,r in enumerate(rs):
213         duration=math.ceil(r['duration'])
214         recharge=math.ceil(charge(r['soc'],m.types[r['g']]['charge']))
215         start=model.new_int_var(0,horizon,f's{j}')
216         end=model.new_int_var(0,horizon+duration,f'e{j}')
217         intervals.append(model.new_interval_var(start,duration,end,f'u{j}'))
218         bend=model.new_int_var(0,horizon+duration+recharge,f'b{j}')
219         battery_intervals.append(model.new_interval_var(start,duration+recharge,bend,f'b{j}'))
220         starts.append(start);ends.append(end)
221         model.add_hint(start,math.ceil(r['start']))
222         for b,t in r['deliver'].items():
223             b=int(b);box=m.bboxes[b]
224             if box['deadline']<1e8:model.add(start<=math.floor(box['deadline']-t))
225             delay=model.new_int_var(0,horizon*1000,f'late{j}_{b}')
226             model.add_max_equality(delay,[0,1000*start+round(t*1000)-round(box['due']*1000)])
227             if not box['medical']:tardy_terms.append(box['priority']*delay)
228             arrival_terms.append(box['priority']*(1000*start+round(t*1000)))
229     for g in m.types:
230         js=[]
231         for j,r in enumerate(rs) if r['g']==g:
232             model.add_cumulative([intervals[j] for j in js],[1]*len(js),len(m.units[g]))
233             model.add_cumulative([battery_intervals[j] for j in js],[1]*len(js),m.types[g]['batteries
                '])
234     cmax=model.new_int_var(0,horizon+max(math.ceil(r['duration']) for r in rs),'cmax')
235     model.add_max_equality(cmax,ends)
236     tardiness=sum(tardy_terms);arrival=sum(arrival_terms)
237     solver=cp_model.CpSolver();solver.parameters.num_search_workers=1;solver.parameters.
        random_seed=optimization_parameters()['method_comparison']['cp_sat_seed']
238     stage=[];chosen=None
239     for label,objective in [('weighted_tardiness',tardiness),('makespan',cmax),('weighted_arrival
        ',arrival)]:
240         model.minimize(objective)
241         solver.parameters.max_time_in_seconds=max(1.0,seconds/3)
242         status=solver.solve(model)
243         info=dict(stage=label,status=solver.status_name(status))
244         if status not in (cp_model.OPTIMAL,cp_model.FEASIBLE):
245             stage.append(info);break
246         chosen=[solver.value(s) for s in starts]
247         info.update(objective=solver.objective_value,bound=solver.best_objective_bound)
248         stage.append(info)
249         model.add(objective<=round(solver.objective_value))
250     if chosen is None:return None,stage
251     out=[]
252     for r,start in zip(rs,chosen):
253         q=dict(r);q.update(start=float(start),end=float(start)+r['duration']);out.append(q)
254     for g in m.types:
255         units=[0.]*len(m.units[g]);batteries=[0.]*m.types[g]['batteries']
256         for r in sorted([z for z in out if z['g']==g],key=lambda z:z['start']):
257             available_units=[k for k,t in enumerate(units) if t<=r['start']+1e-6]
258             available_batteries=[k for k,t in enumerate(batteries) if t<=r['start']+1e-6]
259             if not available_units or not available_batteries:return None,stage
260             ui=available_units[0];bi=available_batteries[0]
261             r['unit']=m.units[g][ui];r['battery']=f'{g}B{bi+1:02}'
262             units[ui]=r['end'];batteries[bi]=r['end']+charge(r['soc'],m.types[g]['charge'])
263     return out,stage
```

A.3 问题三：连续通信链路认证核心代码

以下节选来自通信认证模块，展示单区间保守链路证明，以及认证失败时递归二分路线区间的实际实现。

A.3.1 基于 DEM 扫掠视线的区间证书

```
19 def certificate(m,a,b,fixed,threshold):
20     """证明移动端点沿线段 [a,b] 运动时始终能与 fixed 建立链路。
21
22     距离上界取线段两端中的最大值；遮挡部分使用 10 dB 最坏损耗，或通过
23     栅格化扫掠三角形给出保守视距证明。局部切平面距离额外放大 0.02%。
24     """
25     factor=optimization_parameters()['communication_certificate']['distance_safety_factor']
26     ds=[math.hypot(m.geo.inv(p[0],p[1],fixed[0],fixed[1])[2]*factor,p[2]-fixed[2]) for p in [a,b]]
27     fspl=32.45+20*math.log10(m.comm['frequency_mhz'])+20*math.log10(max(max(ds)/1000,1e-6))
28     if fspl+m.comm['obstruction_db']<=threshold: return threshold-fspl-m.comm['obstruction_db'],'worst_obstruction'
29     if fspl>threshold: return -1., 'distance'
30     a=np.array(a);b=np.array(b);f=np.array(fixed)
31     ab=b[:2]-a[:2];af=f[:2]-a[:2]
32     det=ab[0]*af[1]-ab[1]*af[0]
33     if abs(det)<1e-15:
34         # 仅垂直移动时，较低端点的射线覆盖较高端点对应的更严格遮挡情况。
35         if np.linalg.norm(ab)<1e-12:
36             low=a if a[2]<=b[2] else b
37             return (threshold-fspl,'vertical_LOS') if not m.blocked(low,f) else (-1.,'blocked')
38         return -1., 'degenerate'
39     # 将视距扫掠三角形覆盖到所有相交栅格：平面在每个完整栅格内的最低值
40     # 可作为该栅格内所有实际射线高度的保守下界。
41     xy=np.array([a[:2],b[:2],f[:2]])
42     cols=(xy[:,0]-m.tf.c)/m.tf.a;rows_=(xy[:,1]-m.tf.f)/m.tf.e
43     c0=max(0,int(math.floor(cols.min()))-1);c1=min(m.dem.shape[1],int(math.ceil(cols.max()))+1)
44     r0=max(0,int(math.floor(rows_.min()))-1);r1=min(m.dem.shape[0],int(math.ceil(rows_.max()))+1)
45     tf=m.tf*Affine.translation(c0,r0)
46     geom={'type':'Polygon','coordinates':[[a[:2].tolist(),b[:2].tolist(),f[:2].tolist(),a[:2].tolist()]]}
47     mask=rasterize([(geom,1)],out_shape=(r1-r0,c1-c0),transform=tf,all_touched=True).astype(bool)
48     rr,cc=np.where(mask)
49     slopes=np.linalg.solve(np.array([ab,af]),np.array([b[2]-a[2],f[2]-a[2]]))
50     lon=m.tf.c+(c0+cc+.5)*m.tf.a;lat=m.tf.f+(r0+rr+.5)*m.tf.e
51     z=a[2]+slopes[0]*(lon-a[0])+slopes[1]*(lat-a[1])
52     z-=abs(slopes[0]*m.tf.a)/2+abs(slopes[1]*m.tf.e)/2
53     clear=np.all(m.dem[r0+rr,c0+cc]<=z+1e-8)
54     return (threshold-fspl,'swept_LOS') if clear else (-1.,'unproven')
```

A.3.2 中继时窗筛选与递归区间认证

```
103 def certify_routes(m,rs,relays,verbose=True):
104     gw=gateway(m);records=[];fails=[]
105     for ri,r in enumerate(rs):
106         for si,s in enumerate(r['segments']):
107             def split(f0,f1,depth=0):
108                 a=np.array(s['a']);b=np.array(s['b']);p=a+(b-a)*f0;q=a+(b-a)*f1
109                 t0=r['start']+s['start']+(s['end']-s['start'])*f0
110                 t1=r['start']+s['start']+(s['end']-s['start'])*f1
111                 options=[('G01',gw,m.comm['thresholds_db']['direct'],None)]+[(v['unit'],v['pos'],
m.comm['thresholds_db']['access'],v['id']) for v in relays if t0>=v['ready']-1e-7 and t1<=v['
service_end']+1e-7]
112                 best=None
113                 for provider,fixed,threshold,mission in options:
114                     margin,method=certificate(m,p,q,fixed,threshold)
115                     if margin>=0:
116                         if best is None or margin>best[0]:best=(margin,method,provider,mission)
117                         # 直连裕量至少为 1 dB 时，无需分配中继。
118                         if provider=='G01' and margin>=1:break
119                 if best is not None:
120                     margin,method,provider,mission=best
121                     records.append(dict(route=r['id'],phase=s['phase'],start=t0,end=t1,provider=
provider,relay_task=mission,margin=margin,method=method));return
122                     if depth<optimization_parameters()['communication_certificate']['max_depth']:
123                         mid=(f0+f1)/2;split(f0,mid,depth+1);split(mid,f1,depth+1)
124                     else:fails.append(dict(route=r['id'],phase=s['phase'],time=[t0,t1],pos=((p+q)/2).
tolist()))
125                     dur=s['end']-s['start'];cuts=list(np.linspace(0,1,max(1,math.ceil(dur/
optimization_parameters()['communication_certificate']['step_s']))+1))
126                     if dur>0:
127                         for v in relays:
128                             for edge in [v['ready'],v['service_end']]:
129                                 frac=(edge-r['start']-s['start'])/dur
130                                 if 0<frac<1:cuts.append(frac)
131                     cuts=sorted(set(cuts))
132                     for f0,f1 in zip(cuts[:-1],cuts[1:]):split(f0,f1)
133                 if verbose:print('certified',ri+1,'/',len(rs),'fail',len(fails),flush=True)
134     return records,fails
```

A.4 问题四：不可拆组件分区与资源评价核心代码

以下为项目实际分区函数的完整源码。它先合并多服务区航线形成不可拆组件，再统计各组运输机、电池和中继资源峰值，最后穷举两组与三组划分并按词典序目标筛选。

```
14 def partition(model, data, output="q4.json"):
15     """穷举路线组件在 2 组和 3 组之间的分配，并保存资源需求结果。"""
16     routes = data["routes"]
17     relays = data["relays"]
18     communication = data["communication"]
19
20     # 同一条多服务区路线上的服务区不可拆分，先用并查集构造原子组件。
21     parent = list(range(16))
22
23     def find(node):
24         while parent[node] != node:
25             parent[node] = parent[parent[node]]
26             node = parent[node]
27         return node
28
29     for route in routes:
30         first = route["order"][0]
31         for node in route["order"][1:]:
32             parent[find(node)] = find(first)
33
34     components_by_root = defaultdict(list)
35     for node in range(1, 16):
36         components_by_root[find(node)].append(node)
37     components = list(components_by_root.values())
38     component_count = len(components)
39     print("partition components", components, flush=True)
40
41     inventory = [len(model.units[g]) for g in model.types] + [model.types[g]['batteries'] for g
42     in model.types] + [len(model.relay['units']), model.relay['components']]
43
44     @lru_cache(None)
45     def group(mask):
46         nodes = [
47             node
48             for index, component in enumerate(components)
49             if mask >> index & 1
50             for node in component
51         ]
52         tasks = [route for route in routes if route["order"][0] in nodes]
53         transport_ids = {route["id"] for route in tasks}
54         relay_ids = {
55             interval["relay_task"]
56             for interval in communication
57             if interval["route"] in transport_ids and interval["relay_task"]
58         }
59         relay_tasks = [relay for relay in relays if relay["id"] in relay_ids]
60
61         resources = []
62         for drone_type in model.types:
63             resources.append(
64                 peak(
65                     [
66                         (route["start"], route["end"])
67                         for route in tasks
68                         if route["g"] == drone_type
69                     ]
70                 )
71             )
72         for drone_type in model.types:
73             resources.append(
74                 peak(
75                     [
76                         route["start"],
77                         route["end"]
78                         + charge(route["soc"], model.types[drone_type]["charge"]),
```

```

79         )
80         for route in tasks
81             if route["g"] == drone_type
82                 ]
83             )
84         )
85         resources.append(
86             peak([(relay["start"], relay["end"] + model.relay['turnaround']) for relay in
87                 relay_tasks])
88         )
89         resources.append(
90             peak(
91                 [
92                     (relay["start"], relay["end"] + charge(relay["soc"], model.relay['charge']))
93                     for relay in relay_tasks
94                 ]
95             )
96         )
97         return dict(
98             nodes=nodes,
99             resources=resources,
100             work=sum(route["duration"] for route in tasks),
101             boxes=sum(len(route["boxes"]) for route in tasks),
102             transport_ids=sorted(transport_ids),
103             relay_ids=sorted(relay_ids),
104         )
105
106 all_mask = (1 << component_count) - 1
107 # 统一资源池在原联合时序下的峰值需求是“不分组”基准。
108 # 分组后各组独立配置的总和减该基准，才是重复配置冲余；
109 # 库存减独立配置总和的正部分则是库存闲置，两者不再混用。
110 pooled = group(all_mask) ["resources"]
111 answer = {}
112 for group_count in (2, 3):
113     best = None
114     balanced = None
115     evaluated = 0
116     frontier = {}
117
118     def visit(index, masks):
119         nonlocal best, balanced, evaluated
120         if index == component_count:
121             if len(masks) != group_count:
122                 return
123             evaluated += 1
124             groups = [group(mask) for mask in masks]
125             total = np.sum([item["resources"] for item in groups], axis=0).astype(int).tolist
126
127             deficit = [max(0, target - available) for target, available in zip(total,
128                 inventory)]
129             duplicated = [max(0, target - base) for target, base in zip(total, pooled)]
130             inventory_idle = [max(0, available - target) for target, available in zip(total,
131                 inventory)]
132             workloads = [item["work"] for item in groups]
133             cv = float(np.std(workloads) / np.mean(workloads))
134             score = (sum(deficit), sum(total), cv)
135             record = dict(
136                 K=group_count,
137                 groups=groups,
138                 total=total,
139                 pooled=pooled,
140                 duplicated=duplicated,
141                 deficit=deficit,
142                 inventory_idle=inventory_idle,
143                 unused=inventory_idle, # 保留旧字段以兼容现有出图与提交脚本。
144                 cv=cv,
145                 score=score,
146             )
147             if best is None or score < tuple(best["score"]):
148                 best = record
149             balance_score = (sum(deficit), cv, sum(total))
150             if balanced is None or balance_score < (

```

```

147         sum(balanced["deficit"]), balanced["cv"], sum(balanced["total"]))
148     ):
149         balanced = record
150     key = (sum(deficit), sum(total))
151     if key not in frontier or cv < frontier[key]["cv"]:
152         frontier[key] = record
153     return
154
155     # 递归时将下一个原子组件放入现有小组，或新建一个小组。
156     for group_index in range(len(masks)):
157         next_masks = masks.copy()
158         next_masks[group_index] |= 1 << index
159         visit(index + 1, next_masks)
160     if len(masks) < group_count:
161         visit(index + 1, masks + [1 << index])
162
163 visit(0, [])
164 if best is None:
165     raise RuntimeError("不可拆分组件数量不足，无法组成要求数量的救援小组")
166 answer[str(group_count)] = dict(
167     selected=best,
168     balanced=balanced,
169     evaluated=evaluated,
170     frontier=list(frontier.values()),
171 )
172 print(
173     "Q4",
174     group_count,
175     "evaluated",
176     evaluated,
177     "resources",
178     best["total"],
179     "deficit",
180     best["deficit"],
181     "cv",
182     best["cv"],
183     flush=True,
184 )
185
186 result=dict(components=components, inventory=inventory, pooled=pooled, schemes=answer)
187 save(output, result)
188 return result

```